(continued from previous page)

```
    }
    System.out.println("\n");
    return status;
  }

}
```

2.1.5 Python Examples

This section includes source code for all of the Gurobi Python examples. The same source code can be found in the `examples/python` directory of the Gurobi distribution.

acopf_4buses.py

```
#!/usr/bin/env python

# Copyright 2024, Gurobi Optimization, LLC

# A Power Flow Problem is illustrated on a 4 bus (node) network using an AC (Alternating␣
↪Current) represention
#
#
# Zij = Impedance between i and j                          P_i      = Active Power␣
↪Injected into bus i
# Yij = Admittance between i and j  (Yij = 1/Zij)          Q_i      = Reactive Power␣
↪Injeced into bus i
# Yij = Gij + j Bij    (j = sqrt(-1))                      V_i      = Voltage Magnitude␣
↪at bus i
#                                                        Theta_i  = Voltage Angle at␣
↪bus i
#
# Real (P) and Reactive (Q) Power balance equations must be met on all buses
#
# P_i = V_i \sum_{j=1}^{N} V_j (G_{ij} \cos(\theta_i - \theta_j) + B_{ij} \sin(\theta_i -
↪ \theta_j))
#
# Q_i = V_i \sum_{j=1}^{N} V_j (G_{ij} \sin(\theta_i - \theta_j) - B_{ij} \cos(\theta_i -
↪ \theta_j))
#
#
#   (1)---(2)    Bus 1: Swing Bus. Fixed Voltage Magnitude and Angle. Variable P and Q.
#    |     |     Bus 2: Load Bus.  Fixed P and Q. Variable Voltage Magnitude and Angle.
#    |     |     Bus 3. Generation bus. Fixed Voltage Magnitude and P. Variable Voltage␣
↪Angle and Q.
#   (4)---(3)    Bus 4. Load Bus.  Fixed P and Q. Variable Voltage and Angle.
#
#  Objective: minimize overall reactive power on buses 1 and 3
import numpy as np
import gurobipy as gp
from gurobipy import GRB
```

(continues on next page)

(continued from previous page)

```
from gurobipy import nlfunc


# Number of Buses (Nodes)
N = 4

# Conductance/susceptance components
G = np.array(
    [
        [1.7647, -0.5882, 0.0, -1.1765],
        [-0.5882, 1.5611, -0.3846, -0.5882],
        [0.0, -0.3846, 1.5611, -1.1765],
        [-1.1765, -0.5882, -1.1765, 2.9412],
    ]
)
B = np.array(
    [
        [-7.0588, 2.3529, 0.0, 4.7059],
        [2.3529, -6.629, 1.9231, 2.3529],
        [0.0, 1.9231, -6.629, 4.7059],
        [4.7059, 2.3529, 4.7059, -11.7647],
    ]
)

# Assign bounds where fixings are needed
v_lb = np.array([1.0, 0.0, 1.0, 0.0])
v_ub = np.array([1.0, 1.5, 1.0, 1.5])
P_lb = np.array([-3.0, -0.3, 0.3, -0.2])
P_ub = np.array([3.0, -0.3, 0.3, -0.2])
Q_lb = np.array([-3.0, -0.2, -3.0, -0.15])
Q_ub = np.array([3.0, -0.2, 3.0, -0.15])
theta_lb = np.array([0.0, -np.pi / 2, -np.pi / 2, -np.pi / 2])
theta_ub = np.array([0.0, np.pi / 2, np.pi / 2, np.pi / 2])

with gp.Env() as env, gp.Model("OptimalPowerFlow", env=env) as model:
    P = model.addMVar(N, name="P", lb=P_lb, ub=P_ub)  # Real power for buses
    Q = model.addMVar(N, name="Q", lb=Q_lb, ub=Q_ub)  # Reactive for buses
    v = model.addMVar(N, name="v", lb=v_lb, ub=v_ub)  # Voltage magnitude at buses

    # Voltage angle at buses. The MVar is reshaped to a column vector to
    # simplify the outer subtraction used in power balance constraints.
    theta = model.addMVar(N, name="theta", lb=theta_lb, ub=theta_ub).reshape((N, 1))

    # Minimize Reactive Power at buses 1 and 3
    model.setObjective(Q[[0, 2]].sum(), GRB.MINIMIZE)

    # Real power balance
    constr_P = model.addGenConstrNL(
        P,
        v * ((G * nlfunc.cos(theta - theta.T) + B * nlfunc.sin(theta - theta.T)) @ v),
        name="constr_P",
    )
```

(continues on next page)

(continued from previous page)

```
    # Reactive power balance
    constr_Q = model.addGenConstrNL(
        Q,
        v * ((G * nlfunc.sin(theta - theta.T) - B * nlfunc.cos(theta - theta.T)) @ v),
        name="constr_Q",
    )

    model.optimize()

    # Print output table if pandas is installed
    try:
        import pandas as pd

        df = pd.DataFrame(
            {
                "Bus": range(1, N + 1),
                "Voltage": v.X,
                "Angle": theta.reshape(N).X * 180 / np.pi,
                "Real Power": P.X,
                "Reactive Power": Q.X,
            }
        )

        print(df)
    except ImportError:
        pass
```

batchmode.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example reads a MIP model from a file, solves it in batch mode,
# and prints the JSON solution string.
#
# You will need a Compute Server license for this example to work.

import sys
import time
import json
import gurobipy as gp
from gurobipy import GRB


# Set up the environment for batch mode optimization.
#
# The function creates an empty environment, sets all necessary parameters,
# and returns the ready-to-be-started Env object to caller.   It is the
```

(continues on next page)

(continued from previous page)

```
# caller's responsibility to dispose of this environment when it's no
# longer needed.
def setupbatchenv():
    env = gp.Env(empty=True)
    env.setParam("LogFile", "batchmode.log")
    env.setParam("CSManager", "http://localhost:61080")
    env.setParam("UserName", "gurobi")
    env.setParam("ServerPassword", "pass")
    env.setParam("CSBatchMode", 1)

    # No network communication happened up to this point.  This will happen
    # once the caller invokes the start() method of the returned Env object.

    return env


# Print batch job error information, if any
def printbatcherrorinfo(batch):
    if batch is None or batch.BatchErrorCode == 0:
        return

    print(
        f"Batch ID {batch.BatchID}: Error code {batch.BatchErrorCode} ({batch.
↪BatchErrorMessage})"
    )


# Create a batch request for given problem file
def newbatchrequest(filename):
    # Start environment, create Model object from file
    #
    # By using the context handlers for env and model, it is ensured that
    # model.dispose() and env.dispose() are called automatically
    with setupbatchenv().start() as env, gp.read(filename, env=env) as model:
        # Set some parameters
        model.Params.MIPGap = 0.01
        model.Params.JSONSolDetail = 1

        # Define tags for some variables in order to access their values later
        for count, v in enumerate(model.getVars()):
            v.VTag = f"Variable{count}"
            if count >= 10:
                break

        # Submit batch request
        batchID = model.optimizeBatch()

    return batchID


# Wait for the final status of the batch.
# Initially the status of a batch is "submitted"; the status will change
```

(continues on next page)

(continued from previous page)

```
# once the batch has been processed (by a compute server).
def waitforfinalstatus(batchID):
    # Wait no longer than one hour
    maxwaittime = 3600

    # Setup and start environment, create local Batch handle object
    with setupbatchenv().start() as env, gp.Batch(batchID, env) as batch:
        starttime = time.time()
        while batch.BatchStatus == GRB.BATCH_SUBMITTED:
            # Abort this batch if it is taking too long
            curtime = time.time()
            if curtime - starttime > maxwaittime:
                batch.abort()
                break

            # Wait for two seconds
            time.sleep(2)

            # Update the resident attribute cache of the Batch object with the
            # latest values from the cluster manager.
            batch.update()

            # If the batch failed, we retry it
            if batch.BatchStatus == GRB.BATCH_FAILED:
                batch.retry()

        # Print information about error status of the job that processed the batch
        printbatcherrorinfo(batch)


def printfinalreport(batchID):
    # Setup and start environment, create local Batch handle object
    with setupbatchenv().start() as env, gp.Batch(batchID, env) as batch:
        if batch.BatchStatus == GRB.BATCH_CREATED:
            print("Batch status is 'CREATED'")
        elif batch.BatchStatus == GRB.BATCH_SUBMITTED:
            print("Batch is 'SUBMITTED")
        elif batch.BatchStatus == GRB.BATCH_ABORTED:
            print("Batch is 'ABORTED'")
        elif batch.BatchStatus == GRB.BATCH_FAILED:
            print("Batch is 'FAILED'")
        elif batch.BatchStatus == GRB.BATCH_COMPLETED:
            print("Batch is 'COMPLETED'")
            print("JSON solution:")
            # Get JSON solution as string, create dict from it
            sol = json.loads(batch.getJSONSolution())

            # Pretty printing the general solution information
            print(json.dumps(sol["SolutionInfo"], indent=4))

            # Write the full JSON solution string to a file
            batch.writeJSONSolution("batch-sol.json.gz")
```

(continues on next page)

(continued from previous page)

```
        else:
            # Should not happen
            print("Batch has unknown BatchStatus")

        printbatcherrorinfo(batch)


# Instruct the cluster manager to discard all data relating to this BatchID
def batchdiscard(batchID):
    # Setup and start environment, create local Batch handle object
    with setupbatchenv().start() as env, gp.Batch(batchID, env) as batch:
        # Remove batch request from manager
        batch.discard()


# Solve a given model using batch optimization
if __name__ == "__main__":
    # Ensure we have an input file
    if len(sys.argv) < 2:
        print(f"Usage: {sys.argv[0]} filename")
        sys.exit(0)

    # Submit new batch request
    batchID = newbatchrequest(sys.argv[1])

    # Wait for final status
    waitforfinalstatus(batchID)

    # Report final status info
    printfinalreport(batchID)

    # Remove batch request from manager
    batchdiscard(batchID)

    print("Batch optimization OK")
```

bilinear.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example formulates and solves the following simple bilinear model:
#  maximize    x
#  subject to  x + y + z <= 10
#              x * y <= 2         (bilinear inequality)
#              x * z + y * z = 1  (bilinear equality)
#              x, y, z non-negative (x integral in second version)

import gurobipy as gp
```

(continues on next page)

(continued from previous page)

```
from gurobipy import GRB

# Create a new model
m = gp.Model("bilinear")

# Create variables
x = m.addVar(name="x")
y = m.addVar(name="y")
z = m.addVar(name="z")

# Set objective: maximize x
m.setObjective(1.0 * x, GRB.MAXIMIZE)

# Add linear constraint: x + y + z <= 10
m.addConstr(x + y + z <= 10, "c0")

# Add bilinear inequality constraint: x * y <= 2
m.addConstr(x * y <= 2, "bilinear0")

# Add bilinear equality constraint: x * z + y * z == 1
m.addConstr(x * z + y * z == 1, "bilinear1")

# Optimize model
m.optimize()

m.printAttr("x")

# Constrain 'x' to be integral and solve again
x.VType = GRB.INTEGER
m.optimize()

m.printAttr("x")
```

callback.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

#   This example reads a model from a file, sets up a callback that
#   monitors optimization progress and implements a custom
#   termination strategy, and outputs progress information to the
#   screen and to a log file.
#
#   The termination strategy implemented in this callback stops the
#   optimization of a MIP model once at least one of the following two
#   conditions have been satisfied:
#     1) The optimality gap is less than 10%
#     2) At least 10000 nodes have been explored, and an integer feasible
#        solution has been found.
```

(continues on next page)

(continued from previous page)

```
#   Note that termination is normally handled through Gurobi parameters
#   (MIPGap, NodeLimit, etc.).  You should only use a callback for
#   termination if the available parameters don't capture your desired
#   termination criterion.

import sys
from functools import partial

import gurobipy as gp
from gurobipy import GRB


class CallbackData:
    def __init__(self, modelvars):
        self.modelvars = modelvars
        self.lastiter = -GRB.INFINITY
        self.lastnode = -GRB.INFINITY


def mycallback(model, where, *, cbdata, logfile):
    """
    Callback function. 'model' and 'where' arguments are passed by gurobipy
    when the callback is invoked. The other arguments must be provided via
    functools.partial:
      1) 'cbdata' is an instance of CallbackData, which holds the model
         variables and tracks state information across calls to the callback.
      2) 'logfile' is a writeable file handle.
    """

    if where == GRB.Callback.POLLING:
        # Ignore polling callback
        pass
    elif where == GRB.Callback.PRESOLVE:
        # Presolve callback
        cdels = model.cbGet(GRB.Callback.PRE_COLDEL)
        rdels = model.cbGet(GRB.Callback.PRE_ROWDEL)
        if cdels or rdels:
            print(f"{cdels} columns and {rdels} rows are removed")
    elif where == GRB.Callback.SIMPLEX:
        # Simplex callback
        itcnt = model.cbGet(GRB.Callback.SPX_ITRCNT)
        if itcnt - cbdata.lastiter >= 100:
            cbdata.lastiter = itcnt
            obj = model.cbGet(GRB.Callback.SPX_OBJVAL)
            ispert = model.cbGet(GRB.Callback.SPX_ISPERT)
            pinf = model.cbGet(GRB.Callback.SPX_PRIMINF)
            dinf = model.cbGet(GRB.Callback.SPX_DUALINF)
            if ispert == 0:
                ch = " "
            elif ispert == 1:
                ch = "S"
            else:
```

(continues on next page)

(continued from previous page)

```
            ch = "P"
        print(f"{int(itcnt)} {obj:g}{ch} {pinf:g} {dinf:g}")
elif where == GRB.Callback.MIP:
    # General MIP callback
    nodecnt = model.cbGet(GRB.Callback.MIP_NODCNT)
    objbst = model.cbGet(GRB.Callback.MIP_OBJBST)
    objbnd = model.cbGet(GRB.Callback.MIP_OBJBND)
    solcnt = model.cbGet(GRB.Callback.MIP_SOLCNT)
    if nodecnt - cbdata.lastnode >= 100:
        cbdata.lastnode = nodecnt
        actnodes = model.cbGet(GRB.Callback.MIP_NODLFT)
        itcnt = model.cbGet(GRB.Callback.MIP_ITRCNT)
        cutcnt = model.cbGet(GRB.Callback.MIP_CUTCNT)
        print(
            f"{nodecnt:.0f} {actnodes:.0f} {itcnt:.0f} {objbst:g} "
            f"{objbnd:g} {solcnt} {cutcnt}"
        )
    if abs(objbst - objbnd) < 0.1 * (1.0 + abs(objbst)):
        print("Stop early - 10% gap achieved")
        model.terminate()
    if nodecnt >= 10000 and solcnt:
        print("Stop early - 10000 nodes explored")
        model.terminate()
elif where == GRB.Callback.MIPSOL:
    # MIP solution callback
    nodecnt = model.cbGet(GRB.Callback.MIPSOL_NODCNT)
    obj = model.cbGet(GRB.Callback.MIPSOL_OBJ)
    solcnt = model.cbGet(GRB.Callback.MIPSOL_SOLCNT)
    x = model.cbGetSolution(cbdata.modelvars)
    print(
        f"**** New solution at node {nodecnt:.0f}, obj {obj:g}, "
        f"sol {solcnt:.0f}, x[0] = {x[0]:g} ****"
    )
elif where == GRB.Callback.MIPNODE:
    # MIP node callback
    print("**** New node ****")
    if model.cbGet(GRB.Callback.MIPNODE_STATUS) == GRB.OPTIMAL:
        x = model.cbGetNodeRel(cbdata.modelvars)
        model.cbSetSolution(cbdata.modelvars, x)
elif where == GRB.Callback.BARRIER:
    # Barrier callback
    itcnt = model.cbGet(GRB.Callback.BARRIER_ITRCNT)
    primobj = model.cbGet(GRB.Callback.BARRIER_PRIMOBJ)
    dualobj = model.cbGet(GRB.Callback.BARRIER_DUALOBJ)
    priminf = model.cbGet(GRB.Callback.BARRIER_PRIMINF)
    dualinf = model.cbGet(GRB.Callback.BARRIER_DUALINF)
    cmpl = model.cbGet(GRB.Callback.BARRIER_COMPL)
    print(f"{itcnt:.0f} {primobj:g} {dualobj:g} {priminf:g} {dualinf:g} {cmpl:g}")
elif where == GRB.Callback.MESSAGE:
    # Message callback
    msg = model.cbGet(GRB.Callback.MSG_STRING)
    logfile.write(msg)
```

(continues on next page)

(continued from previous page)

```
# Parse arguments
if len(sys.argv) < 2:
    print("Usage: callback.py filename")
    sys.exit(0)
model_file = sys.argv[1]

# This context block manages several resources to ensure they are properly
# closed at the end of the program:
#   1) A Gurobi environment, with console output and heuristics disabled.
#   2) A Gurobi model, read from a file provided by the user.
#   3) A Python file handle which the callback will write to.

with gp.Env(params={"OutputFlag": 0, "Heuristics": 0}) as env, gp.read(
    model_file, env=env
) as model, open("cb.log", "w") as logfile:

    # Set up callback function with required arguments
    callback_data = CallbackData(model.getVars())
    callback_func = partial(mycallback, cbdata=callback_data, logfile=logfile)

    # Solve model and print solution information
    model.optimize(callback_func)

    print("")
    print("Optimization complete")
    if model.SolCount == 0:
        print(f"No solution found, optimization status = {model.Status}")
    else:
        print(f"Solution found, objective = {model.ObjVal:g}")
        for v in model.getVars():
            if v.X != 0.0:
                print(f"{v.VarName} {v.X:g}")
```

dense.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example formulates and solves the following simple QP model:
#
#    minimize    x + y + x^2 + x*y + y^2 + y*z + z^2
#    subject to  x + 2 y + 3 z >= 4
#                x +   y       >= 1
#                x, y, z non-negative
#
# The example illustrates the use of dense matrices to store A and Q
# (and dense vectors for the other relevant data).  We don't recommend
```

(continues on next page)

(continued from previous page)

```
# that you use dense matrices, but this example may be helpful if you
# already have your data in this format.

import sys
import gurobipy as gp
from gurobipy import GRB


def dense_optimize(rows, cols, c, Q, A, sense, rhs, lb, ub, vtype, solution):
    model = gp.Model()

    # Add variables to model
    vars = []
    for j in range(cols):
        vars.append(model.addVar(lb=lb[j], ub=ub[j], vtype=vtype[j]))

    # Populate A matrix
    for i in range(rows):
        expr = gp.LinExpr()
        for j in range(cols):
            if A[i][j] != 0:
                expr += A[i][j] * vars[j]
        model.addLConstr(expr, sense[i], rhs[i])

    # Populate objective
    obj = gp.QuadExpr()
    for i in range(cols):
        for j in range(cols):
            if Q[i][j] != 0:
                obj += Q[i][j] * vars[i] * vars[j]
    for j in range(cols):
        if c[j] != 0:
            obj += c[j] * vars[j]
    model.setObjective(obj)

    # Solve
    model.optimize()

    # Write model to a file
    model.write("dense.lp")

    if model.status == GRB.OPTIMAL:
        x = model.getAttr("X", vars)
        for i in range(cols):
            solution[i] = x[i]
        return True
    else:
        return False


# Put model data into dense matrices
```

(continues on next page)

(continued from previous page)

```
c = [1, 1, 0]
Q = [[1, 1, 0], [0, 1, 1], [0, 0, 1]]
A = [[1, 2, 3], [1, 1, 0]]
sense = [GRB.GREATER_EQUAL, GRB.GREATER_EQUAL]
rhs = [4, 1]
lb = [0, 0, 0]
ub = [GRB.INFINITY, GRB.INFINITY, GRB.INFINITY]
vtype = [GRB.CONTINUOUS, GRB.CONTINUOUS, GRB.CONTINUOUS]
sol = [0] * 3

# Optimize

success = dense_optimize(2, 3, c, Q, A, sense, rhs, lb, ub, vtype, sol)

if success:
    print(f"x: {sol[0]:g}, y: {sol[1]:g}, z: {sol[2]:g}")
```

diet.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Solve the classic diet model, showing how to add constraints
# to an existing model.

import gurobipy as gp
from gurobipy import GRB


# Nutrition guidelines, based on
# USDA Dietary Guidelines for Americans, 2005
# http://www.health.gov/DietaryGuidelines/dga2005/

categories, minNutrition, maxNutrition = gp.multidict(
    {
        "calories": [1800, 2200],
        "protein": [91, GRB.INFINITY],
        "fat": [0, 65],
        "sodium": [0, 1779],
    }
)

foods, cost = gp.multidict(
    {
        "hamburger": 2.49,
        "chicken": 2.89,
        "hot dog": 1.50,
        "fries": 1.89,
        "macaroni": 2.09,
```

(continues on next page)

(continued from previous page)

```
        "pizza": 1.99,
        "salad": 2.49,
        "milk": 0.89,
        "ice cream": 1.59,
    }
)

# Nutrition values for the foods
nutritionValues = {
    ("hamburger", "calories"): 410,
    ("hamburger", "protein"): 24,
    ("hamburger", "fat"): 26,
    ("hamburger", "sodium"): 730,
    ("chicken", "calories"): 420,
    ("chicken", "protein"): 32,
    ("chicken", "fat"): 10,
    ("chicken", "sodium"): 1190,
    ("hot dog", "calories"): 560,
    ("hot dog", "protein"): 20,
    ("hot dog", "fat"): 32,
    ("hot dog", "sodium"): 1800,
    ("fries", "calories"): 380,
    ("fries", "protein"): 4,
    ("fries", "fat"): 19,
    ("fries", "sodium"): 270,
    ("macaroni", "calories"): 320,
    ("macaroni", "protein"): 12,
    ("macaroni", "fat"): 10,
    ("macaroni", "sodium"): 930,
    ("pizza", "calories"): 320,
    ("pizza", "protein"): 15,
    ("pizza", "fat"): 12,
    ("pizza", "sodium"): 820,
    ("salad", "calories"): 320,
    ("salad", "protein"): 31,
    ("salad", "fat"): 12,
    ("salad", "sodium"): 1230,
    ("milk", "calories"): 100,
    ("milk", "protein"): 8,
    ("milk", "fat"): 2.5,
    ("milk", "sodium"): 125,
    ("ice cream", "calories"): 330,
    ("ice cream", "protein"): 8,
    ("ice cream", "fat"): 10,
    ("ice cream", "sodium"): 180,
}

# Model
m = gp.Model("diet")

# Create decision variables for the foods to buy
buy = m.addVars(foods, name="buy")
```

(continues on next page)

(continued from previous page)

```
# You could use Python looping constructs and m.addVar() to create
# these decision variables instead.  The following would be equivalent
#
# buy = {}
# for f in foods:
#   buy[f] = m.addVar(name=f)

# The objective is to minimize the costs
m.setObjective(buy.prod(cost), GRB.MINIMIZE)

# Using looping constructs, the preceding statement would be:
#
# m.setObjective(sum(buy[f]*cost[f] for f in foods), GRB.MINIMIZE)

# Nutrition constraints
m.addConstrs(
    (
        gp.quicksum(nutritionValues[f, c] * buy[f] for f in foods)
        == [minNutrition[c], maxNutrition[c]]
        for c in categories
    ),
    "_",
)

# Using looping constructs, the preceding statement would be:
#
# for c in categories:
#  m.addRange(sum(nutritionValues[f, c] * buy[f] for f in foods),
#             minNutrition[c], maxNutrition[c], c)


def printSolution():
    if m.status == GRB.OPTIMAL:
        print(f"\nCost: {m.ObjVal:g}")
        print("\nBuy:")
        for f in foods:
            if buy[f].X > 0.0001:
                print(f"{f} {buy[f].X:g}")
    else:
        print("No solution")


# Solve
m.optimize()
printSolution()

print("\nAdding constraint: at most 6 servings of dairy")
m.addConstr(buy.sum(["milk", "ice cream"]) <= 6, "limit_dairy")

# Solve
m.optimize()
```

(continues on next page)

(continued from previous page)

```
printSolution()
```

diet2.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Separate the model (dietmodel.py) from the data file (diet2.py), so
# that the model can be solved with different data files.
#
# Nutrition guidelines, based on
# USDA Dietary Guidelines for Americans, 2005
# http://www.health.gov/DietaryGuidelines/dga2005/

import dietmodel
import gurobipy as gp
from gurobipy import GRB


categories, minNutrition, maxNutrition = gp.multidict(
    {
        "calories": [1800, 2200],
        "protein": [91, GRB.INFINITY],
        "fat": [0, 65],
        "sodium": [0, 1779],
    }
)

foods, cost = gp.multidict(
    {
        "hamburger": 2.49,
        "chicken": 2.89,
        "hot dog": 1.50,
        "fries": 1.89,
        "macaroni": 2.09,
        "pizza": 1.99,
        "salad": 2.49,
        "milk": 0.89,
        "ice cream": 1.59,
    }
)

# Nutrition values for the foods
nutritionValues = {
    ("hamburger", "calories"): 410,
    ("hamburger", "protein"): 24,
    ("hamburger", "fat"): 26,
    ("hamburger", "sodium"): 730,
    ("chicken", "calories"): 420,
```

(continues on next page)

(continued from previous page)

```
    ("chicken", "protein"): 32,
    ("chicken", "fat"): 10,
    ("chicken", "sodium"): 1190,
    ("hot dog", "calories"): 560,
    ("hot dog", "protein"): 20,
    ("hot dog", "fat"): 32,
    ("hot dog", "sodium"): 1800,
    ("fries", "calories"): 380,
    ("fries", "protein"): 4,
    ("fries", "fat"): 19,
    ("fries", "sodium"): 270,
    ("macaroni", "calories"): 320,
    ("macaroni", "protein"): 12,
    ("macaroni", "fat"): 10,
    ("macaroni", "sodium"): 930,
    ("pizza", "calories"): 320,
    ("pizza", "protein"): 15,
    ("pizza", "fat"): 12,
    ("pizza", "sodium"): 820,
    ("salad", "calories"): 320,
    ("salad", "protein"): 31,
    ("salad", "fat"): 12,
    ("salad", "sodium"): 1230,
    ("milk", "calories"): 100,
    ("milk", "protein"): 8,
    ("milk", "fat"): 2.5,
    ("milk", "sodium"): 125,
    ("ice cream", "calories"): 330,
    ("ice cream", "protein"): 8,
    ("ice cream", "fat"): 10,
    ("ice cream", "sodium"): 180,
}

dietmodel.solve(categories, minNutrition, maxNutrition, foods, cost, nutritionValues)
```

diet3.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Use a SQLite database with the diet model (dietmodel.py).  The database
# (diet.db) can be recreated using the included SQL script (diet.sql).
#
# Note that this example reads an external data file (..\data\diet.db).
# As a result, it must be run from the Gurobi examples/python directory.

import os
import sqlite3
import dietmodel
```

(continues on next page)

(continued from previous page)

```
import gurobipy as gp


con = sqlite3.connect(os.path.join("..", "data", "diet.db"))
cur = con.cursor()

cur.execute("select category,minnutrition,maxnutrition from categories")
result = cur.fetchall()
categories, minNutrition, maxNutrition = gp.multidict(
    (cat, [minv, maxv]) for cat, minv, maxv in result
)

cur.execute("select food,cost from foods")
result = cur.fetchall()
foods, cost = gp.multidict(result)

cur.execute("select food,category,value from nutrition")
result = cur.fetchall()
nutritionValues = dict(((f, c), v) for f, c, v in result)

con.close()

dietmodel.solve(categories, minNutrition, maxNutrition, foods, cost, nutritionValues)
```

diet4.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Read diet model data from an Excel spreadsheet (diet.xlsx).
# Pass the imported data into the diet model (dietmodel.py).
#
# Note that this example reads an external data file (..\data\diet.xlsx).
# As a result, it must be run from the Gurobi examples/python directory.
#
# This example uses Python package 'openpyxl', which isn't included
# in most Python distributions.  You can install it with
# 'pip install openpyxl'.

import os
import openpyxl
import dietmodel

# Open 'diet.xlsx'
book = openpyxl.load_workbook(os.path.join("..", "data", "diet.xlsx"))

# Read min/max nutrition info from 'Categories' sheet
sheet = book["Categories"]
categories = []
```

(continues on next page)

(continued from previous page)

```
minNutrition = {}
maxNutrition = {}
for row in sheet.iter_rows():
    category = row[0].value
    if category != "Categories":
        categories.append(category)
        minNutrition[category] = row[1].value
        maxNutrition[category] = row[2].value

# Read food costs from 'Foods' sheet
sheet = book["Foods"]
foods = []
cost = {}
for row in sheet.iter_rows():
    food = row[0].value
    if food != "Foods":
        foods.append(food)
        cost[food] = row[1].value

# Read food nutrition info from 'Nutrition' sheet
sheet = book["Nutrition"]
nutritionValues = {}
for row in sheet.iter_rows():
    if row[0].value == None:  # column labels - categories
        cats = [v.value for v in row]
    else:  # nutrition values
        food = row[0].value
        for col in range(1, len(row)):
            nutritionValues[food, cats[col]] = row[col].value


dietmodel.solve(categories, minNutrition, maxNutrition, foods, cost, nutritionValues)
```

dietmodel.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Solve the classic diet model.  This file implements
# a function that formulates and solves the model,
# but it contains no model data.  The data is
# passed in by the calling program.  Run example 'diet2.py',
# 'diet3.py', or 'diet4.py' to invoke this function.

import gurobipy as gp
from gurobipy import GRB


def solve(categories, minNutrition, maxNutrition, foods, cost, nutritionValues):
```

(continues on next page)

(continued from previous page)

```
    # Model
    m = gp.Model("diet")

    # Create decision variables for the foods to buy
    buy = m.addVars(foods, name="buy")

    # The objective is to minimize the costs
    m.setObjective(buy.prod(cost), GRB.MINIMIZE)

    # Nutrition constraints
    m.addConstrs(
        (
            gp.quicksum(nutritionValues[f, c] * buy[f] for f in foods)
            == [minNutrition[c], maxNutrition[c]]
            for c in categories
        ),
        "_",
    )

    def printSolution():
        if m.status == GRB.OPTIMAL:
            print("\nCost: %g" % m.ObjVal)
            print("\nBuy:")
            for f in foods:
                if buy[f].X > 0.0001:
                    print(f"{f} {buy[f].X:g}")
        else:
            print("No solution")

    # Solve
    m.optimize()
    printSolution()

    print("\nAdding constraint: at most 6 servings of dairy")
    m.addConstr(buy.sum(["milk", "ice cream"]) <= 6, "limit_dairy")

    # Solve
    m.optimize()
    printSolution()
```

facility.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Facility location: a company currently ships its product from 5 plants
# to 4 warehouses. It is considering closing some plants to reduce
# costs. What plant(s) should the company close, in order to minimize
# transportation and fixed costs?
```

(continues on next page)

(continued from previous page)

```
#
# Note that this example uses lists instead of dictionaries.  Since
# it does not work with sparse data, lists are a reasonable option.
#
# Based on an example from Frontline Systems:
#   http://www.solver.com/disfacility.htm
# Used with permission.

import gurobipy as gp
from gurobipy import GRB


# Warehouse demand in thousands of units
demand = [15, 18, 14, 20]

# Plant capacity in thousands of units
capacity = [20, 22, 17, 19, 18]

# Fixed costs for each plant
fixedCosts = [12000, 15000, 17000, 13000, 16000]

# Transportation costs per thousand units
transCosts = [
    [4000, 2000, 3000, 2500, 4500],
    [2500, 2600, 3400, 3000, 4000],
    [1200, 1800, 2600, 4100, 3000],
    [2200, 2600, 3100, 3700, 3200],
]

# Range of plants and warehouses
plants = range(len(capacity))
warehouses = range(len(demand))

# Model
m = gp.Model("facility")

# Plant open decision variables: open[p] == 1 if plant p is open.
open = m.addVars(plants, vtype=GRB.BINARY, obj=fixedCosts, name="open")

# Transportation decision variables: transport[w,p] captures the
# optimal quantity to transport to warehouse w from plant p
transport = m.addVars(warehouses, plants, obj=transCosts, name="trans")

# You could use Python looping constructs and m.addVar() to create
# these decision variables instead.  The following would be equivalent
# to the preceding two statements...
#
# open = []
# for p in plants:
#     open.append(m.addVar(vtype=GRB.BINARY,
#                          obj=fixedCosts[p],
#                          name="open[%d]" % p))
```

(continues on next page)

(continued from previous page)

```
#
# transport = []
# for w in warehouses:
#     transport.append([])
#     for p in plants:
#         transport[w].append(m.addVar(obj=transCosts[w][p],
#                                      name="trans[%d,%d]" % (w, p)))

# The objective is to minimize the total fixed and variable costs
m.ModelSense = GRB.MINIMIZE

# Production constraints
# Note that the right-hand limit sets the production to zero if the plant
# is closed
m.addConstrs(
    (transport.sum("*", p) <= capacity[p] * open[p] for p in plants), "Capacity"
)

# Using Python looping constructs, the preceding would be...
#
# for p in plants:
#     m.addConstr(sum(transport[w][p] for w in warehouses)
#                 <= capacity[p] * open[p], "Capacity[%d]" % p)

# Demand constraints
m.addConstrs((transport.sum(w) == demand[w] for w in warehouses), "Demand")

# ... and the preceding would be ...
# for w in warehouses:
#     m.addConstr(sum(transport[w][p] for p in plants) == demand[w],
#                 "Demand[%d]" % w)

# Save model
m.write("facilityPY.lp")

# Guess at the starting point: close the plant with the highest fixed costs;
# open all others

# First open all plants
for p in plants:
    open[p].Start = 1.0

# Now close the plant with the highest fixed cost
print("Initial guess:")
maxFixed = max(fixedCosts)
for p in plants:
    if fixedCosts[p] == maxFixed:
        open[p].Start = 0.0
        print(f"Closing plant {p}")
        break
print("")
```

(continues on next page)

(continued from previous page)

```
# Use barrier to solve root relaxation
m.Params.Method = 2

# Solve
m.optimize()

# Print solution
print(f"\nTOTAL COSTS: {m.ObjVal:g}")
print("SOLUTION:")
for p in plants:
    if open[p].X > 0.99:
        print(f"Plant {p} open")
        for w in warehouses:
            if transport[w, p].X > 0:
                print(f"  Transport {transport[w, p].X:g} units to warehouse {w}")
    else:
        print(f"Plant {p} closed!")
```

feasopt.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example reads a MIP model from a file, adds artificial
# variables to each constraint, and then minimizes the sum of the
# artificial variables.  A solution with objective zero corresponds
# to a feasible solution to the input model.
#
# We can also use FeasRelax feature to do it. In this example, we
# use minrelax=1, i.e. optimizing the returned model finds a solution
# that minimizes the original objective, but only from among those
# solutions that minimize the sum of the artificial variables.

import sys
import gurobipy as gp

if len(sys.argv) < 2:
    print("Usage: feasopt.py filename")
    sys.exit(0)

feasmodel = gp.read(sys.argv[1])

# create a copy to use FeasRelax feature later

feasmodel1 = feasmodel.copy()

# clear objective

feasmodel.setObjective(0.0)
```

(continues on next page)

(continued from previous page)

```
# add slack variables

for c in feasmodel.getConstrs():
    sense = c.Sense
    if sense != ">":
        feasmodel.addVar(
            obj=1.0, name=f"ArtN_{c.ConstrName}", column=gp.Column([-1], [c])
        )
    if sense != "<":
        feasmodel.addVar(
            obj=1.0, name=f"ArtP_{c.ConstrName}", column=gp.Column([1], [c])
        )

# optimize modified model

feasmodel.optimize()

feasmodel.write("feasopt.lp")

# use FeasRelax feature

feasmodel1.feasRelaxS(0, True, False, True)

feasmodel1.write("feasopt1.lp")

feasmodel1.optimize()
```

fixanddive.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Implement a simple MIP heuristic.  Relax the model,
# sort variables based on fractionality, and fix the 25% of
# the fractional variables that are closest to integer variables.
# Repeat until either the relaxation is integer feasible or
# linearly infeasible.

import sys
import gurobipy as gp
from gurobipy import GRB


# Key function used to sort variables based on relaxation fractionality


def sortkey(v1):
    sol = v1.X
```

(continues on next page)

(continued from previous page)

```
    return abs(sol - int(sol + 0.5))


if len(sys.argv) < 2:
    print("Usage: fixanddive.py filename")
    sys.exit(0)

# Read model

model = gp.read(sys.argv[1])

# Collect integer variables and relax them
intvars = []
for v in model.getVars():
    if v.VType != GRB.CONTINUOUS:
        intvars += [v]
        v.VType = GRB.CONTINUOUS

model.Params.OutputFlag = 0

model.optimize()


# Perform multiple iterations.  In each iteration, identify the first
# quartile of integer variables that are closest to an integer value in the
# relaxation, fix them to the nearest integer, and repeat.

for iter in range(1000):
    # create a list of fractional variables, sorted in order of increasing
    # distance from the relaxation solution to the nearest integer value

    fractional = []
    for v in intvars:
        sol = v.X
        if abs(sol - int(sol + 0.5)) > 1e-5:
            fractional += [v]

    fractional.sort(key=sortkey)

    print(f"Iteration {iter}, obj {model.ObjVal:g}, fractional {len(fractional)}")

    if len(fractional) == 0:
        print(f"Found feasible solution - objective {model.ObjVal:g}")
        break

    # Fix the first quartile to the nearest integer value
    nfix = max(int(len(fractional) / 4), 1)
    for i in range(nfix):
        v = fractional[i]
        fixval = int(v.X + 0.5)
        v.LB = fixval
        v.UB = fixval
```

(continues on next page)

(continued from previous page)

```
        print(f"  Fix {v.VarName} to {fixval:g} (rel {v.X:g})")

    model.optimize()

    # Check optimization result

    if model.Status != GRB.OPTIMAL:
        print("Relaxation is infeasible")
        break
```

gc_funcnonlinear.py

```
#!/usr/bin/env python3.11

# Copyright 2024, Gurobi Optimization, LLC

# This example considers the following nonconvex nonlinear problem
#
#  minimize   sin(x) + cos(2*x) + 1
#  subject to  0.25*exp(x) - x <= 0
#              -1 <= x <= 4
#
#  We show you two approaches to solve it as a nonlinear model:
#
#  1) Set the paramter FuncNonlinear = 1 to handle all general function
#     constraints as true nonlinear functions.
#
#  2) Set the attribute FuncNonlinear = 1 for each general function
#     constraint to handle these as true nonlinear functions.
#

import gurobipy as gp
from gurobipy import GRB


def printsol(m, x):
    print(f"x = {x.X}")
    print(f"Obj = {m.ObjVal}")


try:
    # Create a new model
    m = gp.Model()

    # Create variables
    x = m.addVar(lb=-1, ub=4, name="x")
    twox = m.addVar(lb=-2, ub=8, name="2x")
    sinx = m.addVar(lb=-1, ub=1, name="sinx")
    cos2x = m.addVar(lb=-1, ub=1, name="cos2x")
    expx = m.addVar(name="expx")
```

(continues on next page)

(continued from previous page)

```
    # Set objective
    m.setObjective(sinx + cos2x + 1, GRB.MINIMIZE)

    # Add linear constraints
    lc1 = m.addConstr(0.25 * expx - x <= 0)
    lc2 = m.addConstr(2.0 * x - twox == 0)

    # Add general function constraints
    # sinx = sin(x)
    gc1 = m.addGenConstrSin(x, sinx, "gc1")
    # cos2x = cos(twox)
    gc2 = m.addGenConstrCos(twox, cos2x, "gc2")
    # expx = exp(x)
    gc3 = m.addGenConstrExp(x, expx, "gc3")

    # Approach 1) Set FuncNonlinear parameter

    m.params.FuncNonlinear = 1

    # Optimize the model
    m.optimize()

    printsol(m, x)

    # Restore unsolved state and set parameter FuncNonlinear to
    # its default value
    m.reset()
    m.resetParams()

    # Approach 2) Set FuncNonlinear attribute for every
    #             general function constraint

    gc1.FuncNonlinear = 1
    gc2.FuncNonlinear = 1
    gc3.FuncNonlinear = 1

    m.optimize()

    printsol(m, x)

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

gc_pwl.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example formulates and solves the following simple model
# with PWL constraints:
#
#  maximize
#        sum c[j] * x[j]
#  subject to
#        sum A[i,j] * x[j] <= 0,  for i = 0, ..., m-1
#        sum y[j] <= 3
#        y[j] = pwl(x[j]),        for j = 0, ..., n-1
#        x[j] free, y[j] >= 0,    for j = 0, ..., n-1
#  where pwl(x) = 0,     if x  = 0
#               = 1+|x|, if x != 0
#
#  Note
#   1. sum pwl(x[j]) <= b is to bound x vector and also to favor sparse x vector.
#      Here b = 3 means that at most two x[j] can be nonzero and if two, then
#      sum x[j] <= 1
#   2. pwl(x) jumps from 1 to 0 and from 0 to 1, if x moves from negative 0 to 0,
#      then to positive 0, so we need three points at x = 0. x has infinite bounds
#      on both sides, the piece defined with two points (-1, 2) and (0, 1) can
#      extend x to -infinite. Overall we can use five points (-1, 2), (0, 1),
#      (0, 0), (0, 1) and (1, 2) to define y = pwl(x)
#

import gurobipy as gp
from gurobipy import GRB

try:
    n = 5
    m = 5
    c = [0.5, 0.8, 0.5, 0.1, -1]
    A = [
        [0, 0, 0, 1, -1],
        [0, 0, 1, 1, -1],
        [1, 1, 0, 0, -1],
        [1, 0, 1, 0, -1],
        [1, 0, 0, 1, -1],
    ]

    # Create a new model
    model = gp.Model("gc_pwl")

    # Create variables
    x = model.addVars(n, lb=-GRB.INFINITY, name="x")
    y = model.addVars(n, name="y")

    # Set objective
```

(continues on next page)

(continued from previous page)

```
    model.setObjective(gp.quicksum(c[j] * x[j] for j in range(n)), GRB.MAXIMIZE)

    # Add Constraints
    for i in range(m):
        model.addConstr(gp.quicksum(A[i][j] * x[j] for j in range(n)) <= 0)

    model.addConstr(y.sum() <= 3)

    for j in range(n):
        model.addGenConstrPWL(x[j], y[j], [-1, 0, 0, 0, 1], [2, 1, 0, 1, 2])

    # Optimize model
    model.optimize()

    for j in range(n):
        print(f"{x[j].VarName} = {x[j].X:g}")

    print(f"Obj: {model.ObjVal:g}")

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

gc_pwl_func.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example considers the following nonconvex nonlinear problem
#
#  maximize    2 x    + y
#  subject to  exp(x) + 4 sqrt(y) <= 9
#              x, y >= 0
#
#  We show you two approaches to solve this:
#
#  1) Use a piecewise-linear approach to handle general function
#     constraints (such as exp and sqrt).
#     a) Add two variables
#        u = exp(x)
#        v = sqrt(y)
#     b) Compute points (x, u) of u = exp(x) for some step length (e.g., x
#        = 0, 1e-3, 2e-3, ..., xmax) and points (y, v) of v = sqrt(y) for
#        some step length (e.g., y = 0, 1e-3, 2e-3, ..., ymax). We need to
#        compute xmax and ymax (which is easy for this example, but this
#        does not hold in general).
#     c) Use the points to add two general constraints of type
```

(continues on next page)

(continued from previous page)

```
#        piecewise-linear.
#
#  2) Use the Gurobis built-in general function constraints directly (EXP
#     and POW). Here, we do not need to compute the points and the maximal
#     possible values, which will be done internally by Gurobi.  In this
#     approach, we show how to "zoom in" on the optimal solution and
#     tighten tolerances to improve the solution quality.
#

import math
import gurobipy as gp
from gurobipy import GRB


def printsol(m, x, y, u, v):
    print(f"x = {x.X}, u = {u.X}")
    print(f"y = {y.X}, v = {v.X}")
    print(f"Obj = {m.ObjVal}")

    # Calculate violation of exp(x) + 4 sqrt(y) <= 9
    vio = math.exp(x.X) + 4 * math.sqrt(y.X) - 9
    if vio < 0:
        vio = 0
    print(f"Vio = {vio}")


try:
    # Create a new model
    m = gp.Model()

    # Create variables
    x = m.addVar(name="x")
    y = m.addVar(name="y")
    u = m.addVar(name="u")
    v = m.addVar(name="v")

    # Set objective
    m.setObjective(2 * x + y, GRB.MAXIMIZE)

    # Add constraints
    lc = m.addConstr(u + 4 * v <= 9)

    # Approach 1) PWL constraint approach

    xpts = []
    ypts = []
    upts = []
    vpts = []

    intv = 1e-3

    xmax = math.log(9)
```

(continues on next page)

(continued from previous page)

```
    t = 0.0
    while t < xmax + intv:
        xpts.append(t)
        upts.append(math.exp(t))
        t += intv

    ymax = (9.0 / 4) * (9.0 / 4)
    t = 0.0
    while t < ymax + intv:
        ypts.append(t)
        vpts.append(math.sqrt(t))
        t += intv

    gc1 = m.addGenConstrPWL(x, u, xpts, upts, "gc1")
    gc2 = m.addGenConstrPWL(y, v, ypts, vpts, "gc2")

    # Optimize the model
    m.optimize()

    printsol(m, x, y, u, v)

    # Approach 2) General function constraint approach with auto PWL
    #             translation by Gurobi

    # restore unsolved state and get rid of PWL constraints
    m.reset()
    m.remove(gc1)
    m.remove(gc2)
    m.update()

    # u = exp(x)
    gcf1 = m.addGenConstrExp(x, u, name="gcf1")
    # v = y^(0.5)
    gcf2 = m.addGenConstrPow(y, v, 0.5, name="gcf2")

    # Use the equal piece length approach with the length = 1e-3
    m.Params.FuncPieces = 1
    m.Params.FuncPieceLength = 1e-3

    # Optimize the model
    m.optimize()

    printsol(m, x, y, u, v)

    # Zoom in, use optimal solution to reduce the ranges and use a smaller
    # pclen=1-5 to solve it

    x.LB = max(x.LB, x.X - 0.01)
    x.UB = min(x.UB, x.X + 0.01)
    y.LB = max(y.LB, y.X - 0.01)
    y.UB = min(y.UB, y.X + 0.01)
    m.update()
```

(continues on next page)

(continued from previous page)

```
    m.reset()

    m.Params.FuncPieceLength = 1e-5

    # Optimize the model
    m.optimize()

    printsol(m, x, y, u, v)

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

genconstr.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# In this example we show the use of general constraints for modeling
# some common expressions. We use as an example a SAT-problem where we
# want to see if it is possible to satisfy at least four (or all) clauses
# of the logical form
#
# L = (x0 or ~x1 or x2)  and (x1 or ~x2 or x3)  and
#     (x2 or ~x3 or x0)  and (x3 or ~x0 or x1)  and
#     (~x0 or ~x1 or x2) and (~x1 or ~x2 or x3) and
#     (~x2 or ~x3 or x0) and (~x3 or ~x0 or x1)
#
# We do this by introducing two variables for each literal (itself and its
# negated value), one variable for each clause, one variable indicating
# whether we can satisfy at least four clauses, and one last variable to
# identify the minimum of the clauses (so if it is one, we can satisfy all
# clauses). Then we put these last two variables in the objective.
# The objective function is therefore
#
# maximize Obj0 + Obj1
#
#  Obj0 = MIN(Clause1, ... , Clause8)
#  Obj1 = 1 -> Clause1 + ... + Clause8 >= 4
#
# thus, the objective value will be two if and only if we can satisfy all
# clauses; one if and only if at least four but not all clauses can be satisfied,
# and zero otherwise.

import gurobipy as gp
from gurobipy import GRB
import sys
```

(continues on next page)

(continued from previous page)

```
try:
    NLITERALS = 4

    n = NLITERALS

    # Example data:
    #   e.g. {0, n+1, 2} means clause (x0 or ~x1 or x2)
    Clauses = [
        [0, n + 1, 2],
        [1, n + 2, 3],
        [2, n + 3, 0],
        [3, n + 0, 1],
        [n + 0, n + 1, 2],
        [n + 1, n + 2, 3],
        [n + 2, n + 3, 0],
        [n + 3, n + 0, 1],
    ]

    # Create a new model
    model = gp.Model("Genconstr")

    # initialize decision variables and objective
    Lit = model.addVars(NLITERALS, vtype=GRB.BINARY, name="X")
    NotLit = model.addVars(NLITERALS, vtype=GRB.BINARY, name="NotX")

    Cla = model.addVars(len(Clauses), vtype=GRB.BINARY, name="Clause")

    Obj0 = model.addVar(vtype=GRB.BINARY, name="Obj0")
    Obj1 = model.addVar(vtype=GRB.BINARY, name="Obj1")

    # Link Xi and notXi
    model.addConstrs(
        (Lit[i] + NotLit[i] == 1.0 for i in range(NLITERALS)), name="CNSTR_X"
    )

    # Link clauses and literals
    for i, c in enumerate(Clauses):
        clause = []
        for l in c:
            if l >= n:
                clause.append(NotLit[l - n])
            else:
                clause.append(Lit[l])
        model.addConstr(Cla[i] == gp.or_(clause), "CNSTR_Clause" + str(i))

    # Link objs with clauses
    model.addConstr(Obj0 == gp.min_(Cla.values()), name="CNSTR_Obj0")
    model.addConstr((Obj1 == 1) >> (Cla.sum() >= 4.0), name="CNSTR_Obj1")

    # Set optimization objective
    model.setObjective(Obj0 + Obj1, GRB.MAXIMIZE)
```

(continues on next page)

(continued from previous page)

```
    # Save problem
    model.write("genconstr.mps")
    model.write("genconstr.lp")

    # Optimize
    model.optimize()

    # Status checking
    status = model.getAttr(GRB.Attr.Status)

    if status in (GRB.INF_OR_UNBD, GRB.INFEASIBLE, GRB.UNBOUNDED):
        print("The model cannot be solved because it is infeasible or unbounded")
        sys.exit(1)

    if status != GRB.OPTIMAL:
        print("Optimization was stopped with status ", status)
        sys.exit(1)

    # Print result
    objval = model.getAttr(GRB.Attr.ObjVal)

    if objval > 1.9:
        print("Logical expression is satisfiable")
    elif objval > 0.9:
        print("At least four clauses can be satisfied")
    else:
        print("Not even three clauses can be satisfied")

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

genconstrnl.py

```
import gurobipy as gp
from gurobipy import nlfunc


# Formulate and solve the simple nonlinear model

# Minimize y
# s.t.     y = sin(2.5 x1) + x2
#          -1 <= x1, x2 <= 1

with gp.Env() as env, gp.Model(env=env) as model:
    # Optimization variables
    x1 = model.addVar(lb=-1, ub=1, name="x1")
```

(continues on next page)

(continued from previous page)

```
    x2 = model.addVar(lb=-1, ub=1, name="x2")

    # Auxiliary resultant variable for general constraint
    y = model.addVar(lb=-float("inf"), name="y")

    # Nonlinear constraint for y
    model.addGenConstrNL(y, nlfunc.sin(2.5 * x1) + x2)

    # Use y for objective function
    model.setObjective(y)

    model.optimize()

    print(f"x1={x1.X}  x2={x2.X}  obj={y.X}")
```

heurcb.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example uses a callback to implement a simple rounding heuristic.
# It reads a MIP model from a file, sets up the callback, turns off
# the standard heuristics, and then applies the callback heuristic
# at every opportunity.

import sys
import math
import gurobipy as gp
from gurobipy import GRB


# Define callback function


def mycallback(model, where):
    if where == GRB.Callback.MIPNODE:
        if model.cbGet(GRB.Callback.MIPNODE_STATUS) == GRB.OPTIMAL:
            integers = []
            fixval = []
            for v in model.getVars():
                if v.vtype != GRB.CONTINUOUS:
                    x = model.cbGetNodeRel(v)
                    integers += [v]
                    fixval += [math.ceil(x - 1e-5)]  # Round all int vars up
            model.cbSetSolution(integers, fixval)


if len(sys.argv) < 2:
    print("Usage: heurcb.py filename")
```

(continues on next page)

(continued from previous page)

```
    sys.exit(0)

# Read and solve model

model = gp.read(sys.argv[1])

# Turn off Gurobi heuristics

model.setParam("Heuristics", 0)

# Solve model

model.optimize(mycallback)
```

lp.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example reads an LP model from a file and solves it.
# If the model is infeasible or unbounded, the example turns off
# presolve and solves the model again. If the model is infeasible,
# the example computes an Irreducible Inconsistent Subsystem (IIS),
# and writes it to a file

import sys
import gurobipy as gp
from gurobipy import GRB

if len(sys.argv) < 2:
    print("Usage: lp.py filename")
    sys.exit(0)

# Read and solve model

model = gp.read(sys.argv[1])
model.optimize()

if model.Status == GRB.INF_OR_UNBD:
    # Turn presolve off to determine whether model is infeasible
    # or unbounded
    model.setParam(GRB.Param.Presolve, 0)
    model.optimize()

if model.Status == GRB.OPTIMAL:
    print(f"Optimal objective: {model.ObjVal:g}")
    model.write("model.sol")
    sys.exit(0)
elif model.Status != GRB.INFEASIBLE:
```

(continues on next page)

(continued from previous page)

```
    print(f"Optimization was stopped with status {model.Status}")
    sys.exit(0)


# Model is infeasible - compute an Irreducible Inconsistent Subsystem (IIS)

print("")
print("Model is infeasible")
model.computeIIS()
model.write("model.ilp")
print("IIS written to file 'model.ilp'")
```

lpmethod.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Solve a model with different values of the Method parameter;
# show which value gives the shortest solve time.

import sys
import gurobipy as gp
from gurobipy import GRB

if len(sys.argv) < 2:
    print("Usage: lpmethod.py filename")
    sys.exit(0)

# Read model
m = gp.read(sys.argv[1])

# Solve the model with different values of Method
bestTime = m.Params.TimeLimit
bestMethod = -1
for i in range(3):
    m.reset()
    m.Params.Method = i
    m.optimize()
    if m.Status == GRB.OPTIMAL:
        bestTime = m.Runtime
        bestMethod = i
        # Reduce the TimeLimit parameter to save time with other methods
        m.Params.TimeLimit = bestTime

# Report which method was fastest
if bestMethod == -1:
    print("Unable to solve this model")
else:
    print(f"Solved in {bestTime:g} seconds with Method {bestMethod}")
```

lpmod.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example reads an LP model from a file and solves it.
# If the model can be solved, then it finds the smallest positive variable,
# sets its upper bound to zero, and resolves the model two ways:
# first with an advanced start, then without an advanced start
# (i.e. 'from scratch').

import sys
import gurobipy as gp
from gurobipy import GRB

if len(sys.argv) < 2:
    print("Usage: lpmod.py filename")
    sys.exit(0)

# Read model and determine whether it is an LP

model = gp.read(sys.argv[1])
if model.IsMIP == 1:
    print("The model is not a linear program")
    sys.exit(1)

model.optimize()

status = model.Status

if status == GRB.INF_OR_UNBD or status == GRB.INFEASIBLE or status == GRB.UNBOUNDED:
    print("The model cannot be solved because it is infeasible or unbounded")
    sys.exit(1)

if status != GRB.OPTIMAL:
    print(f"Optimization was stopped with status {status}")
    sys.exit(0)

# Find the smallest variable value
minVal = GRB.INFINITY
for v in model.getVars():
    if v.X > 0.0001 and v.X < minVal and v.LB == 0.0:
        minVal = v.X
        minVar = v

print(f"\n*** Setting {minVar.VarName} from {minVal:g} to zero ***\n")
minVar.UB = 0.0

# Solve from this starting point
model.optimize()

# Save iteration & time info
```

(continues on next page)

(continued from previous page)

```
warmCount = model.IterCount
warmTime = model.Runtime

# Reset the model and resolve
print("\n*** Resetting and solving without an advanced start ***\n")
model.reset()
model.optimize()

coldCount = model.IterCount
coldTime = model.Runtime

print("")
print(f"*** Warm start: {warmCount:g} iterations, {warmTime:g} seconds")
print(f"*** Cold start: {coldCount:g} iterations, {coldTime:g} seconds")
```

matrix1.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example formulates and solves the following simple MIP model
# using the matrix API:
#  maximize
#        x +   y + 2 z
#  subject to
#        x + 2 y + 3 z <= 4
#        x +   y       >= 1
#        x, y, z binary

import gurobipy as gp
from gurobipy import GRB
import numpy as np
import scipy.sparse as sp

try:
    # Create a new model
    m = gp.Model("matrix1")

    # Create variables
    x = m.addMVar(shape=3, vtype=GRB.BINARY, name="x")

    # Set objective
    obj = np.array([1.0, 1.0, 2.0])
    m.setObjective(obj @ x, GRB.MAXIMIZE)

    # Build (sparse) constraint matrix
    val = np.array([1.0, 2.0, 3.0, -1.0, -1.0])
    row = np.array([0, 0, 0, 1, 1])
    col = np.array([0, 1, 2, 0, 1])
```

(continues on next page)

(continued from previous page)

```
    A = sp.csr_matrix((val, (row, col)), shape=(2, 3))

    # Build rhs vector
    rhs = np.array([4.0, -1.0])

    # Add constraints
    m.addConstr(A @ x <= rhs, name="c")

    # Optimize model
    m.optimize()

    print(x.X)
    print(f"Obj: {m.ObjVal:g}")

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

matrix2.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example uses the matrix friendly API to formulate the n-queens
# problem; it maximizes the number queens placed on an n x n
# chessboard without threatening each other.
#
# This example demonstrates slicing on MVar objects.

import numpy as np
import gurobipy as gp
from gurobipy import GRB

n = 8

m = gp.Model("nqueens")

# n-by-n binary variables; x[i, j] decides whether a queen is placed at
# position (i, j)
x = m.addMVar((n, n), vtype=GRB.BINARY, name="x")

# Maximize the number of placed queens
m.setObjective(x.sum(), GRB.MAXIMIZE)

# At most one queen per row; this adds n linear constraints
m.addConstr(x.sum(axis=1) <= 1, name="row")
```

(continues on next page)

(continued from previous page)

```
# At most one queen per column; this adds n linear constraints
m.addConstr(x.sum(axis=0) <= 1, name="col")

for i in range(-n + 1, n):
    # At most one queen on diagonal i
    m.addConstr(x.diagonal(i).sum() <= 1, name=f"diag{i:d}")

    # At most one queen on anti-diagonal i
    m.addConstr(x[:, ::-1].diagonal(i).sum() <= 1, name=f"adiag{i:d}")

# Solve the problem
m.optimize()

print(x.X)
print(f"Queens placed: {m.ObjVal:.0f}")
```

mip1.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example formulates and solves the following simple MIP model:
#  maximize
#        x +   y + 2 z
#  subject to
#        x + 2 y + 3 z <= 4
#        x +   y       >= 1
#        x, y, z binary

import gurobipy as gp
from gurobipy import GRB

try:
    # Create a new model
    m = gp.Model("mip1")

    # Create variables
    x = m.addVar(vtype=GRB.BINARY, name="x")
    y = m.addVar(vtype=GRB.BINARY, name="y")
    z = m.addVar(vtype=GRB.BINARY, name="z")

    # Set objective
    m.setObjective(x + y + 2 * z, GRB.MAXIMIZE)

    # Add constraint: x + 2 y + 3 z <= 4
    m.addConstr(x + 2 * y + 3 * z <= 4, "c0")

    # Add constraint: x + y >= 1
```

(continues on next page)

(continued from previous page)

```
    m.addConstr(x + y >= 1, "c1")

    # Optimize model
    m.optimize()

    for v in m.getVars():
        print(f"{v.VarName} {v.X:g}")

    print(f"Obj: {m.ObjVal:g}")

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

mip1_remote.py

```
import gurobipy as gp
from gurobipy import GRB

# Variation of mip1.py, with a focus on remote services
#
# When remote resources are tied to the optimization process, such as a token
# server, compute server, or Instant Cloud, extra care should be taken to
# ensure that such resources are released once they are no longer needed.
# Technically, such resources are managed by a gurobipy.Env object
# ("environment").  This example shows best practices for acquiring and
# releasing such shared resources via Env objects.
#
# See also https://docs.gurobi.com/projects/optimizer/en/current/reference/misc/emptyenv.
↪html#configuration-parameters


def populate_and_solve(m):
    # This function formulates and solves the following MIP model (see mip1.py):
    #  maximize
    #        x +   y + 2 z
    #  subject to
    #        x + 2 y + 3 z <= 4
    #        x +   y       >= 1
    #        x, y, z binary

    # Create variables
    x = m.addVar(vtype=GRB.BINARY, name="x")
    y = m.addVar(vtype=GRB.BINARY, name="y")
    z = m.addVar(vtype=GRB.BINARY, name="z")

    # Set objective
    m.setObjective(x + y + 2 * z, GRB.MAXIMIZE)
```

(continues on next page)

(continued from previous page)

```
    # Add constraint: x + 2 y + 3 z <= 4
    m.addConstr(x + 2 * y + 3 * z <= 4, "c0")

    # Add constraint: x + y >= 1
    m.addConstr(x + y >= 1, "c1")

    # Optimize model
    m.optimize()

    for v in m.getVars():
        print(f"{v.VarName} {v.X:g}")

    print(f"Obj: {m.ObjVal:g}")


# Put any connection parameters for Gurobi Compute Server, Gurobi Cluster
# Manager or Gurobi Token server here, unless they are set already
# through the license file.

connection_params = {
    # For Compute Server you need at least this
    #       "ComputeServer": "<server name>",
    #       "UserName": "<user name>",
    #       "ServerPassword": "<password>",
    # For Cluster Manager you need at least this
    #       "CSManager": "<manager name>",
    #       "CSAPIAccessID": "<access ID>",
    #       "CSAPISecret": "<secret>",
    # For Instant cloud you need at least this
    #       "CloudAccessID": "<access id>",
    #       "CloudSecretKey": "<secret>",
}

with gp.Env(params=connection_params) as env:
    # 'env' is now set up according to the connection parameters.
    # The environment is disposed of automatically through the context manager
    # upon leaving this block.
    with gp.Model(env=env) as model:
        # 'model' is now an instance tied to the enclosing Env object 'env'.
        # The model is disposed of automatically through the context manager
        # upon leaving this block.
        try:
            populate_and_solve(model)
        except:
            # Add appropriate error handling here.
            raise
```

mip2.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example reads a MIP model from a file, solves it and prints
# the objective values from all feasible solutions generated while
# solving the MIP. Then it creates the associated fixed model and
# solves that model.

import sys
import gurobipy as gp
from gurobipy import GRB

if len(sys.argv) < 2:
    print("Usage: mip2.py filename")
    sys.exit(0)

# Read and solve model

model = gp.read(sys.argv[1])

if model.IsMIP == 0:
    print("Model is not a MIP")
    sys.exit(0)

model.optimize()

if model.Status == GRB.OPTIMAL:
    print(f"Optimal objective: {model.ObjVal:g}")
elif model.Status == GRB.INF_OR_UNBD:
    print("Model is infeasible or unbounded")
    sys.exit(0)
elif model.Status == GRB.INFEASIBLE:
    print("Model is infeasible")
    sys.exit(0)
elif model.Status == GRB.UNBOUNDED:
    print("Model is unbounded")
    sys.exit(0)
else:
    print(f"Optimization ended with status {model.Status}")
    sys.exit(0)

# Iterate over the solutions and compute the objectives
model.Params.OutputFlag = 0
print("")
for k in range(model.SolCount):
    model.Params.SolutionNumber = k
    print(f"Solution {k} has objective {model.PoolObjVal:g}")
print("")
model.Params.OutputFlag = 1
```

(continues on next page)

(continued from previous page)

```
fixed = model.fixed()
fixed.Params.Presolve = 0
fixed.optimize()

if fixed.Status != GRB.OPTIMAL:
    print("Error: fixed model isn't optimal")
    sys.exit(1)

diff = model.ObjVal - fixed.ObjVal

if abs(diff) > 1e-6 * (1.0 + abs(model.ObjVal)):
    print("Error: objective values are different")
    sys.exit(1)

# Print values of nonzero variables
for v in fixed.getVars():
    if v.X != 0:
        print(f"{v.VarName} {v.X:g}")
```

multiobj.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Want to cover four different sets but subject to a common budget of
# elements allowed to be used. However, the sets have different priorities to
# be covered; and we tackle this by using multi-objective optimization.

import gurobipy as gp
from gurobipy import GRB
import sys

try:
    # Sample data
    Groundset = range(20)
    Subsets = range(4)
    Budget = 12
    Set = [
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0],
        [0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0],
    ]
    SetObjPriority = [3, 2, 2, 1]
    SetObjWeight = [1.0, 0.25, 1.25, 1.0]

    # Create initial model
    model = gp.Model("multiobj")
```

(continues on next page)

(continued from previous page)

```
    # Initialize decision variables for ground set:
    # x[e] == 1 if element e is chosen for the covering.
    Elem = model.addVars(Groundset, vtype=GRB.BINARY, name="El")

    # Constraint: limit total number of elements to be picked to be at most
    # Budget
    model.addConstr(Elem.sum() <= Budget, name="Budget")

    # Set global sense for ALL objectives
    model.ModelSense = GRB.MAXIMIZE

    # Limit how many solutions to collect
    model.setParam(GRB.Param.PoolSolutions, 100)

    # Set and configure i-th objective
    for i in Subsets:
        objn = sum(Elem[k] * Set[i][k] for k in range(len(Elem)))
        model.setObjectiveN(
            objn, i, SetObjPriority[i], SetObjWeight[i], 1.0 + i, 0.01, "Set" + str(i)
        )

    # Save problem
    model.write("multiobj.lp")

    # Optimize
    model.optimize()

    model.setParam(GRB.Param.OutputFlag, 0)

    # Status checking
    status = model.Status
    if status in (GRB.INF_OR_UNBD, GRB.INFEASIBLE, GRB.UNBOUNDED):
        print("The model cannot be solved because it is infeasible or unbounded")
        sys.exit(1)

    if status != GRB.OPTIMAL:
        print(f"Optimization was stopped with status {status}")
        sys.exit(1)

    # Print best selected set
    print("Selected elements in best solution:")
    selected = [e for e in Groundset if Elem[e].X > 0.9]
    print(" ".join(f"El{e}" for e in selected))

    # Print number of solutions stored
    nSolutions = model.SolCount
    print(f"Number of solutions found: {nSolutions}")

    # Print objective values of solutions
    if nSolutions > 10:
        nSolutions = 10
    print(f"Objective values for first {nSolutions} solutions:")
```

(continues on next page)

(continued from previous page)

```
    for i in Subsets:
        model.setParam(GRB.Param.ObjNumber, i)
        objvals = []
        for e in range(nSolutions):
            model.setParam(GRB.Param.SolutionNumber, e)
            objvals.append(model.ObjNVal)

        print(f"\tSet{i}" + "".join(f" {objval:6g}" for objval in objvals[:3]))

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError as e:
    print(f"Encountered an attribute error: {e}")
```

multiscenario.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Facility location: a company currently ships its product from 5 plants to
# 4 warehouses. It is considering closing some plants to reduce costs. What
# plant(s) should the company close, in order to minimize transportation
# and fixed costs?
#
# Since the plant fixed costs and the warehouse demands are uncertain, a
# scenario approach is chosen.
#
# Note that this example is similar to the facility.py example. Here we
# added scenarios in order to illustrate the multi-scenario feature.
#
# Note that this example uses lists instead of dictionaries.  Since
# it does not work with sparse data, lists are a reasonable option.
#
# Based on an example from Frontline Systems:
#   http://www.solver.com/disfacility.htm
# Used with permission.

import gurobipy as gp
from gurobipy import GRB


# Warehouse demand in thousands of units
demand = [15, 18, 14, 20]

# Plant capacity in thousands of units
capacity = [20, 22, 17, 19, 18]

# Fixed costs for each plant
```

(continues on next page)

(continued from previous page)

```
fixedCosts = [12000, 15000, 17000, 13000, 16000]
maxFixed = max(fixedCosts)
minFixed = min(fixedCosts)

# Transportation costs per thousand units
transCosts = [
    [4000, 2000, 3000, 2500, 4500],
    [2500, 2600, 3400, 3000, 4000],
    [1200, 1800, 2600, 4100, 3000],
    [2200, 2600, 3100, 3700, 3200],
]

# Range of plants and warehouses
plants = range(len(capacity))
warehouses = range(len(demand))

# Model
m = gp.Model("multiscenario")

# Plant open decision variables: open[p] == 1 if plant p is open.
open = m.addVars(plants, vtype=GRB.BINARY, obj=fixedCosts, name="open")

# Transportation decision variables: transport[w,p] captures the
# optimal quantity to transport to warehouse w from plant p
transport = m.addVars(warehouses, plants, obj=transCosts, name="trans")

# You could use Python looping constructs and m.addVar() to create
# these decision variables instead.  The following would be equivalent
# to the preceding two statements...
#
# open = []
# for p in plants:
#     open.append(m.addVar(vtype=GRB.BINARY,
#                          obj=fixedCosts[p],
#                          name="open[%d]" % p))
#
# transport = []
# for w in warehouses:
#     transport.append([])
#     for p in plants:
#         transport[w].append(m.addVar(obj=transCosts[w][p],
#                                      name="trans[%d,%d]" % (w, p)))

# The objective is to minimize the total fixed and variable costs
m.ModelSense = GRB.MINIMIZE

# Production constraints
# Note that the right-hand limit sets the production to zero if the plant
# is closed
m.addConstrs(
    (transport.sum("*", p) <= capacity[p] * open[p] for p in plants), "Capacity"
)
```

(continues on next page)

(continued from previous page)

```
# Using Python looping constructs, the preceding would be...
#
# for p in plants:
#     m.addConstr(sum(transport[w][p] for w in warehouses)
#                 <= capacity[p] * open[p], "Capacity[%d]" % p)

# Demand constraints
demandConstr = m.addConstrs(
    (transport.sum(w) == demand[w] for w in warehouses), "Demand"
)

# ... and the preceding would be ...
# for w in warehouses:
#     m.addConstr(sum(transport[w][p] for p in plants) == demand[w],
#                 "Demand[%d]" % w)

# We constructed the base model, now we add 7 scenarios
#
# Scenario 0: Represents the base model, hence, no manipulations.
# Scenario 1: Manipulate the warehouses demands slightly (constraint right
#             hand sides).
# Scenario 2: Double the warehouses demands (constraint right hand sides).
# Scenario 3: Manipulate the plant fixed costs (objective coefficients).
# Scenario 4: Manipulate the warehouses demands and fixed costs.
# Scenario 5: Force the plant with the largest fixed cost to stay open
#             (variable bounds).
# Scenario 6: Force the plant with the smallest fixed cost to be closed
#             (variable bounds).

m.NumScenarios = 7

# Scenario 0: Base model, hence, nothing to do except giving the scenario a
#             name
m.Params.ScenarioNumber = 0
m.ScenNName = "Base model"

# Scenario 1: Increase the warehouse demands by 10%
m.Params.ScenarioNumber = 1
m.ScenNName = "Increased warehouse demands"
for w in warehouses:
    demandConstr[w].ScenNRhs = demand[w] * 1.1

# Scenario 2: Double the warehouse demands
m.Params.ScenarioNumber = 2
m.ScenNName = "Double the warehouse demands"
for w in warehouses:
    demandConstr[w].ScenNRhs = demand[w] * 2.0

# Scenario 3: Decrease the plant fixed costs by 5%
m.Params.ScenarioNumber = 3
m.ScenNName = "Decreased plant fixed costs"
```

(continues on next page)

(continued from previous page)

```
for p in plants:
    open[p].ScenNObj = fixedCosts[p] * 0.95

# Scenario 4: Combine scenario 1 and scenario 3
m.Params.ScenarioNumber = 4
m.ScenNName = "Increased warehouse demands and decreased plant fixed costs"
for w in warehouses:
    demandConstr[w].ScenNRhs = demand[w] * 1.1
for p in plants:
    open[p].ScenNObj = fixedCosts[p] * 0.95

# Scenario 5: Force the plant with the largest fixed cost to stay open
m.Params.ScenarioNumber = 5
m.ScenNName = "Force plant with largest fixed cost to stay open"
open[fixedCosts.index(maxFixed)].ScenNLB = 1.0

# Scenario 6: Force the plant with the smallest fixed cost to be closed
m.Params.ScenarioNumber = 6
m.ScenNName = "Force plant with smallest fixed cost to be closed"
open[fixedCosts.index(minFixed)].ScenNUB = 0.0

# Save model
m.write("multiscenario.lp")

# Guess at the starting point: close the plant with the highest fixed costs;
# open all others

# First open all plants
for p in plants:
    open[p].Start = 1.0

# Now close the plant with the highest fixed cost
p = fixedCosts.index(maxFixed)
open[p].Start = 0.0
print(f"Initial guess: Closing plant {p}\n")

# Use barrier to solve root relaxation
m.Params.Method = 2

# Solve multi-scenario model
m.optimize()

# Print solution for each scenario
for s in range(m.NumScenarios):
    # Set the scenario number to query the information for this scenario
    m.Params.ScenarioNumber = s

    print(f"\n\n------ Scenario {s} ({m.ScenNName})")

    # Check if we found a feasible solution for this scenario
    if m.ModelSense * m.ScenNObjVal >= GRB.INFINITY:
        if m.ModelSense * m.ScenNObjBound >= GRB.INFINITY:
```

(continues on next page)

(continued from previous page)

```
            # Scenario was proven to be infeasible
            print("\nINFEASIBLE")
        else:
            # We did not find any feasible solution - should not happen in
            # this case, because we did not set any limit (like a time
            # limit) on the optimization process
            print("\nNO SOLUTION")
    else:
        print(f"\nTOTAL COSTS: {m.ScenNObjVal:g}")
        print("SOLUTION:")
        for p in plants:
            if open[p].ScenNX > 0.5:
                print(f"Plant {p} open")
                for w in warehouses:
                    if transport[w, p].ScenNX > 0:
                        print(
                            f"  Transport {transport[w, p].ScenNX:g} units to warehouse
↪{w}"
                        )
            else:
                print(f"Plant {p} closed!")

# Print a summary table: for each scenario we add a single summary line
print("\n\nSummary: Closed plants depending on scenario\n")
print(f"{'':8} | {'Plant':>17} {'|':>13}")

tableStr = [f"{'Scenario':8} |"]
tableStr += [f"{p:>5}" for p in plants]
tableStr += [f"| {'Costs':>6}  Name"]
print(" ".join(tableStr))

for s in range(m.NumScenarios):
    # Set the scenario number to query the information for this scenario
    m.Params.ScenarioNumber = s

    tableStr = f"{s:<8} |"

    # Check if we found a feasible solution for this scenario
    if m.ModelSense * m.ScenNObjVal >= GRB.INFINITY:
        if m.ModelSense * m.ScenNObjBound >= GRB.INFINITY:
            # Scenario was proven to be infeasible
            print(tableStr, f"{'infeasible':<30}| {'-':>6}  {m.ScenNName:<}")
        else:
            # We did not find any feasible solution - should not happen in
            # this case, because we did not set any limit (like a time
            # limit) on the optimization process
            print(tableStr, f"{'no solution found':<30}| {'-':>6}  {m.ScenNName:<}")
    else:
        for p in plants:
            if open[p].ScenNX > 0.5:
                tableStr += f" {' ':>5}"
            else:
```

(continues on next page)

(continued from previous page)

```
                tableStr += f" {'x':>5}"

        print(tableStr, f"| {m.ScenNObjVal:6g}  {m.ScenNName:<}")
```

netflow.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Solve a multi-commodity flow problem.  Two products ('Pencils' and 'Pens')
# are produced in 2 cities ('Detroit' and 'Denver') and must be sent to
# warehouses in 3 cities ('Boston', 'New York', and 'Seattle') to
# satisfy supply/demand ('inflow[h,i]').
#
# Flows on the transportation network must respect arc capacity constraints
# ('capacity[i,j]'). The objective is to minimize the sum of the arc
# transportation costs ('cost[i,j]').

import gurobipy as gp
from gurobipy import GRB

# Base data
commodities = ["Pencils", "Pens"]
nodes = ["Detroit", "Denver", "Boston", "New York", "Seattle"]

arcs, capacity = gp.multidict(
    {
        ("Detroit", "Boston"): 100,
        ("Detroit", "New York"): 80,
        ("Detroit", "Seattle"): 120,
        ("Denver", "Boston"): 120,
        ("Denver", "New York"): 120,
        ("Denver", "Seattle"): 120,
    }
)

# Cost for triplets commodity-source-destination
cost = {
    ("Pencils", "Detroit", "Boston"): 10,
    ("Pencils", "Detroit", "New York"): 20,
    ("Pencils", "Detroit", "Seattle"): 60,
    ("Pencils", "Denver", "Boston"): 40,
    ("Pencils", "Denver", "New York"): 40,
    ("Pencils", "Denver", "Seattle"): 30,
    ("Pens", "Detroit", "Boston"): 20,
    ("Pens", "Detroit", "New York"): 20,
    ("Pens", "Detroit", "Seattle"): 80,
    ("Pens", "Denver", "Boston"): 60,
    ("Pens", "Denver", "New York"): 70,
```

(continues on next page)

(continued from previous page)

```
    ("Pens", "Denver", "Seattle"): 30,
}

# Supply (> 0) and demand (< 0) for pairs of commodity-city
inflow = {
    ("Pencils", "Detroit"): 50,
    ("Pencils", "Denver"): 60,
    ("Pencils", "Boston"): -50,
    ("Pencils", "New York"): -50,
    ("Pencils", "Seattle"): -10,
    ("Pens", "Detroit"): 60,
    ("Pens", "Denver"): 40,
    ("Pens", "Boston"): -40,
    ("Pens", "New York"): -30,
    ("Pens", "Seattle"): -30,
}

# Create optimization model
m = gp.Model("netflow")

# Create variables
flow = m.addVars(commodities, arcs, obj=cost, name="flow")

# Arc-capacity constraints
m.addConstrs((flow.sum("*", i, j) <= capacity[i, j] for i, j in arcs), "cap")

# Equivalent version using Python looping
# for i, j in arcs:
#   m.addConstr(sum(flow[h, i, j] for h in commodities) <= capacity[i, j],
#               "cap[%s, %s]" % (i, j))


# Flow-conservation constraints
m.addConstrs(
    (
        flow.sum(h, "*", j) + inflow[h, j] == flow.sum(h, j, "*")
        for h in commodities
        for j in nodes
    ),
    "node",
)

# Alternate version:
# m.addConstrs(
#   (gp.quicksum(flow[h, i, j] for i, j in arcs.select('*', j)) + inflow[h, j] ==
#     gp.quicksum(flow[h, j, k] for j, k in arcs.select(j, '*'))
#     for h in commodities for j in nodes), "node")

# Compute optimal solution
m.optimize()

# Print solution
```

(continues on next page)

(continued from previous page)

```
if m.Status == GRB.OPTIMAL:
    solution = m.getAttr("X", flow)
    for h in commodities:
        print(f"\nOptimal flows for {h}:")
        for i, j in arcs:
            if solution[h, i, j] > 0:
                print(f"{i} -> {j}: {solution[h, i, j]:g}")
```

params.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Use parameters that are associated with a model.
#
# A MIP is solved for a few seconds with different sets of parameters.
# The one with the smallest MIP gap is selected, and the optimization
# is resumed until the optimal solution is found.

import sys
import gurobipy as gp


if len(sys.argv) < 2:
    print("Usage: params.py filename")
    sys.exit(0)


# Read model and verify that it is a MIP
m = gp.read(sys.argv[1])
if m.IsMIP == 0:
    print("The model is not an integer program")
    sys.exit(1)

# Set a 2 second time limit
m.Params.TimeLimit = 2

# Now solve the model with different values of MIPFocus
bestModel = m.copy()
bestModel.optimize()
for i in range(1, 4):
    m.reset()
    m.Params.MIPFocus = i
    m.optimize()
    if bestModel.MIPGap > m.MIPGap:
        bestModel, m = m, bestModel  # swap models

# Finally, delete the extra model, reset the time limit and
# continue to solve the best model to optimality
```

(continues on next page)

(continued from previous page)

```
del m
bestModel.Params.TimeLimit = float("inf")
bestModel.optimize()
print(f"Solved with MIPFocus: {bestModel.Params.MIPFocus}")
```

piecewise.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example considers the following separable, convex problem:
#
#   minimize    f(x) - y + g(z)
#   subject to  x + 2 y + 3 z <= 4
#               x +   y       >= 1
#               x,    y,    z <= 1
#
# where f(u) = exp(-u) and g(u) = 2 u^2 - 4 u, for all real u. It
# formulates and solves a simpler LP model by approximating f and
# g with piecewise-linear functions. Then it transforms the model
# into a MIP by negating the approximation for f, which corresponds
# to a non-convex piecewise-linear function, and solves it again.

import gurobipy as gp
from math import exp


def f(u):
    return exp(-u)


def g(u):
    return 2 * u * u - 4 * u


try:
    # Create a new model

    m = gp.Model()

    # Create variables

    lb = 0.0
    ub = 1.0

    x = m.addVar(lb, ub, name="x")
    y = m.addVar(lb, ub, name="y")
    z = m.addVar(lb, ub, name="z")
```

(continues on next page)

(continued from previous page)

```
    # Set objective for y

    m.setObjective(-y)

    # Add piecewise-linear objective functions for x and z

    npts = 101
    ptu = []
    ptf = []
    ptg = []

    for i in range(npts):
        ptu.append(lb + (ub - lb) * i / (npts - 1))
        ptf.append(f(ptu[i]))
        ptg.append(g(ptu[i]))

    m.setPWLObj(x, ptu, ptf)
    m.setPWLObj(z, ptu, ptg)

    # Add constraint: x + 2 y + 3 z <= 4

    m.addConstr(x + 2 * y + 3 * z <= 4, "c0")

    # Add constraint: x + y >= 1

    m.addConstr(x + y >= 1, "c1")

    # Optimize model as an LP

    m.optimize()

    print(f"IsMIP: {m.IsMIP}")
    for v in m.getVars():
        print(f"{v.VarName} {v.X:g}")
    print(f"Obj: {m.ObjVal:g}")
    print("")

    # Negate piecewise-linear objective function for x

    for i in range(npts):
        ptf[i] = -ptf[i]

    m.setPWLObj(x, ptu, ptf)

    # Optimize model as a MIP

    m.optimize()

    print(f"IsMIP: {m.IsMIP}")
    for v in m.getVars():
        print(f"{v.VarName} {v.X:g}")
    print(f"Obj: {m.ObjVal:g}")
```

(continues on next page)

(continued from previous page)

```
except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

poolsearch.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# We find alternative epsilon-optimal solutions to a given knapsack
# problem by using PoolSearchMode

from __future__ import print_function
import gurobipy as gp
from gurobipy import GRB
import sys

try:
    # Sample data
    Groundset = range(10)
    objCoef = [32, 32, 15, 15, 6, 6, 1, 1, 1, 1]
    knapsackCoef = [16, 16, 8, 8, 4, 4, 2, 2, 1, 1]
    Budget = 33

    # Create initial model
    model = gp.Model("poolsearch")

    # Create dicts for tupledict.prod() function
    objCoefDict = dict(zip(Groundset, objCoef))
    knapsackCoefDict = dict(zip(Groundset, knapsackCoef))

    # Initialize decision variables for ground set:
    # x[e] == 1 if element e is chosen
    Elem = model.addVars(Groundset, vtype=GRB.BINARY, name="El")

    # Set objective function
    model.ModelSense = GRB.MAXIMIZE
    model.setObjective(Elem.prod(objCoefDict))

    # Constraint: limit total number of elements to be picked to be at most
    # Budget
    model.addConstr(Elem.prod(knapsackCoefDict) <= Budget, name="Budget")

    # Limit how many solutions to collect
    model.setParam(GRB.Param.PoolSolutions, 1024)
    # Limit the search space by setting a gap for the worst possible solution
```

(continues on next page)

(continued from previous page)

```
    # that will be accepted
    model.setParam(GRB.Param.PoolGap, 0.10)
    # do a systematic search for the k-best solutions
    model.setParam(GRB.Param.PoolSearchMode, 2)

    # save problem
    model.write("poolsearch.lp")

    # Optimize
    model.optimize()

    model.setParam(GRB.Param.OutputFlag, 0)

    # Status checking
    status = model.Status
    if status in (GRB.INF_OR_UNBD, GRB.INFEASIBLE, GRB.UNBOUNDED):
        print("The model cannot be solved because it is infeasible or unbounded")
        sys.exit(1)

    if status != GRB.OPTIMAL:
        print(f"Optimization was stopped with status {status}")
        sys.exit(1)

    # Print best selected set
    print("Selected elements in best solution:")
    print("\t", end="")
    for e in Groundset:
        if Elem[e].X > 0.9:
            print(f" El{e}", end="")
    print("")

    # Print number of solutions stored
    nSolutions = model.SolCount
    print(f"Number of solutions found: {nSolutions}")

    # Print objective values of solutions
    for e in range(nSolutions):
        model.setParam(GRB.Param.SolutionNumber, e)
        print(f"{model.PoolObjVal:g} ", end="")
        if e % 15 == 14:
            print("")
    print("")

    # print fourth best set if available
    if nSolutions >= 4:
        model.setParam(GRB.Param.SolutionNumber, 3)

        print("Selected elements in fourth best solution:")
        print("\t", end="")
        for e in Groundset:
            if Elem[e].Xn > 0.9:
                print(f" El{e}", end="")
```

(continues on next page)

(continued from previous page)

```
        print("")

except gp.GurobiError as e:
    print(f"Gurobi error {e.errno}: {e.message}")

except AttributeError as e:
    print(f"Encountered an attribute error: {e}")
```

portfolio.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Portfolio selection: given a sum of money to invest, one must decide how to
# spend it amongst a portfolio of financial securities.  Our approach is due
# to Markowitz (1959) and looks to minimize the risk associated with the
# investment while realizing a target expected return.  By varying the target,
# one can compute an 'efficient frontier', which defines the optimal portfolio
# for a given expected return.
#
# Note that this example reads historical return data from a comma-separated
# file (../data/portfolio.csv).  As a result, it must be run from the Gurobi
# examples/python directory.
#
# This example requires the pandas (>= 0.20.3), NumPy, and Matplotlib
# Python packages, which are part of the SciPy ecosystem for
# mathematics, science, and engineering (http://scipy.org).  These
# packages aren't included in all Python distributions, but are
# included by default with Anaconda Python.

import gurobipy as gp
from gurobipy import GRB
from math import sqrt
import pandas as pd
import numpy as np
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Import (normalized) historical return data using pandas
data = pd.read_csv("../data/portfolio.csv", index_col=0)
stocks = data.columns

# Calculate basic summary statistics for individual stocks
stock_volatility = data.std()
stock_return = data.mean()

# Turn off all logging and create an empty model
```

(continues on next page)

(continued from previous page)

```
with gp.Env(params={"OutputFlag": 0}) as env, gp.Model("portfolio", env=env) as m:

    # Add a variable for each stock
    vars = pd.Series(m.addVars(stocks), index=stocks)

    # Objective is to minimize risk (squared).  This is modeled using the
    # covariance matrix, which measures the historical correlation between stocks.
    sigma = data.cov()
    portfolio_risk = sigma.dot(vars).dot(vars)
    m.setObjective(portfolio_risk, GRB.MINIMIZE)

    # Fix budget with a constraint
    m.addConstr(vars.sum() == 1, "budget")

    # Optimize model to find the minimum risk portfolio
    m.optimize()

    # Create an expression representing the expected return for the portfolio
    portfolio_return = stock_return.dot(vars)

    # Display minimum risk portfolio
    print("Minimum Risk Portfolio:\n")
    for v in vars:
        if v.X > 0:
            print(f"\t{v.VarName}\t: {v.X:g}")
    minrisk_volatility = sqrt(portfolio_risk.getValue())
    print(f"\nVolatility      = {minrisk_volatility:g}")
    minrisk_return = portfolio_return.getValue()
    print(f"Expected Return = {minrisk_return:g}")

    # Add (redundant) target return constraint
    target = m.addConstr(portfolio_return == minrisk_return, "target")

    # Solve for efficient frontier by varying target return
    frontier = pd.Series(dtype=np.float64)
    for r in np.linspace(stock_return.min(), stock_return.max(), 100):
        target.rhs = r
        m.optimize()
        frontier.loc[sqrt(portfolio_risk.getValue())] = r

# Plot volatility versus expected return for individual stocks
ax = plt.gca()
ax.scatter(x=stock_volatility, y=stock_return, color="Blue", label="Individual Stocks")
for stock in stocks:
    ax.annotate(stock, (stock_volatility[stock], stock_return[stock]))

# Plot volatility versus expected return for minimum risk portfolio
ax.scatter(x=minrisk_volatility, y=minrisk_return, color="DarkGreen")
ax.annotate(
    "Minimum\nRisk\nPortfolio",
    (minrisk_volatility, minrisk_return),
    horizontalalignment="right",
```

(continues on next page)

(continued from previous page)

```
)

# Plot efficient frontier
frontier.plot(color="DarkGreen", label="Efficient Frontier", ax=ax)

# Format and display the final plot
ax.axis((0.005, 0.06, -0.02, 0.025))
ax.set_xlabel("Volatility (standard deviation)")
ax.set_ylabel("Expected Return")
ax.legend()
ax.grid()
plt.savefig("portfolio.png")
print("Plotted efficient frontier to 'portfolio.png'")
```

qcp.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example formulates and solves the following simple QCP model:
#  maximize    x
#  subject to  x + y + z = 1
#              x^2 + y^2 <= z^2 (second-order cone)
#              x^2 <= yz        (rotated second-order cone)
#              x, y, z non-negative

import gurobipy as gp
from gurobipy import GRB

# Create a new model
m = gp.Model("qcp")

# Create variables
x = m.addVar(name="x")
y = m.addVar(name="y")
z = m.addVar(name="z")

# Set objective: x
obj = 1.0 * x
m.setObjective(obj, GRB.MAXIMIZE)

# Add constraint: x + y + z = 1
m.addConstr(x + y + z == 1, "c0")

# Add second-order cone: x^2 + y^2 <= z^2
m.addConstr(x**2 + y**2 <= z**2, "qc0")

# Add rotated cone: x^2 <= yz
m.addConstr(x**2 <= y * z, "qc1")
```

(continues on next page)

(continued from previous page)

```
m.optimize()

for v in m.getVars():
    print(f"{v.VarName} {v.X:g}")

print(f"Obj: {m.ObjVal:g}")
```

qp.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example formulates and solves the following simple QP model:
#  minimize
#      x^2 + x*y + y^2 + y*z + z^2 + 2 x
#  subject to
#      x + 2 y + 3 z >= 4
#      x +   y       >= 1
#      x, y, z non-negative
#
# It solves it once as a continuous model, and once as an integer model.

import gurobipy as gp
from gurobipy import GRB

# Create a new model
m = gp.Model("qp")

# Create variables
x = m.addVar(ub=1.0, name="x")
y = m.addVar(ub=1.0, name="y")
z = m.addVar(ub=1.0, name="z")

# Set objective: x^2 + x*y + y^2 + y*z + z^2 + 2 x
obj = x**2 + x * y + y**2 + y * z + z**2 + 2 * x
m.setObjective(obj)

# Add constraint: x + 2 y + 3 z >= 4
m.addConstr(x + 2 * y + 3 * z >= 4, "c0")

# Add constraint: x + y >= 1
m.addConstr(x + y >= 1, "c1")

m.optimize()

for v in m.getVars():
    print(f"{v.VarName} {v.X:g}")
```

(continues on next page)

(continued from previous page)

```
print(f"Obj: {m.ObjVal:g}")

x.VType = GRB.INTEGER
y.VType = GRB.INTEGER
z.VType = GRB.INTEGER

m.optimize()

for v in m.getVars():
    print(f"{v.VarName} {v.X:g}")

print(f"Obj: {m.ObjVal:g}")
```

sensitivity.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# A simple sensitivity analysis example which reads a MIP model from a file
# and solves it. Then uses the scenario feature to analyze the impact
# w.r.t. the objective function of each binary variable if it is set to
# 1-X, where X is its value in the optimal solution.
#
# Usage:
#     sensitivity.py <model filename>
#

import sys
import gurobipy as gp
from gurobipy import GRB


# Maximum number of scenarios to be considered
maxScenarios = 100

if len(sys.argv) < 2:
    print("Usage: sensitivity.py filename")
    sys.exit(0)

# Read model
model = gp.read(sys.argv[1])

if model.IsMIP == 0:
    print("Model is not a MIP")
    sys.exit(0)

# Solve model
model.optimize()
```

(continues on next page)

(continued from previous page)

```
if model.Status != GRB.OPTIMAL:
    print(f"Optimization ended with status {model.Status}")
    sys.exit(0)

# Store the optimal solution
origObjVal = model.ObjVal
for v in model.getVars():
    v._origX = v.X


scenarios = 0

# Count number of unfixed, binary variables in model. For each we create a
# scenario.
for v in model.getVars():
    if v.LB == 0.0 and v.UB == 1.0 and v.VType in (GRB.BINARY, GRB.INTEGER):
        scenarios += 1

        if scenarios >= maxScenarios:
            break


# Set the number of scenarios in the model
model.NumScenarios = scenarios
scenarios = 0

print(f"###  construct multi-scenario model with {scenarios} scenarios")

# Create a (single) scenario model by iterating through unfixed binary
# variables in the model and create for each of these variables a scenario
# by fixing the variable to 1-X, where X is its value in the computed
# optimal solution
for v in model.getVars():
    if (
        v.LB == 0.0
        and v.UB == 1.0
        and v.VType in (GRB.BINARY, GRB.INTEGER)
        and scenarios < maxScenarios
    ):
        # Set ScenarioNumber parameter to select the corresponding scenario
        # for adjustments
        model.Params.ScenarioNumber = scenarios

        # Set variable to 1-X, where X is its value in the optimal solution
        if v._origX < 0.5:
            v.ScenNLB = 1.0
        else:
            v.ScenNUB = 0.0

        scenarios += 1

    else:
```

(continues on next page)

(continued from previous page)

```
        # Add MIP start for all other variables using the optimal solution
        # of the base model
        v.Start = v._origX


# Solve multi-scenario model
model.optimize()


# In case we solved the scenario model to optimality capture the
# sensitivity information
if model.Status == GRB.OPTIMAL:
    modelSense = model.ModelSense
    scenarios = 0

    # Capture sensitivity information from each scenario
    for v in model.getVars():
        if v.LB == 0.0 and v.UB == 1.0 and v.VType in (GRB.BINARY, GRB.INTEGER):
            # Set scenario parameter to collect the objective value of the
            # corresponding scenario
            model.Params.ScenarioNumber = scenarios

            # Collect objective value and bound for the scenario
            scenarioObjVal = model.ScenNObjVal
            scenarioObjBound = model.ScenNObjBound

            # Check if we found a feasible solution for this scenario
            if modelSense * scenarioObjVal >= GRB.INFINITY:
                # Check if the scenario is infeasible
                if modelSense * scenarioObjBound >= GRB.INFINITY:
                    print(
                        f"Objective sensitivity for variable {v.VarName} is infeasible"
                    )
                else:
                    print(
                        f"Objective sensitivity for variable {v.VarName} is unknown (no␣
↪solution available)"
                    )
            else:
                # Scenario is feasible and a solution is available
                print(
                    f"Objective sensitivity for variable {v.VarName} is {modelSense *␣
↪(scenarioObjVal - origObjVal):g}"
                )

            scenarios += 1

            if scenarios >= maxScenarios:
                break
```

sos.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# This example creates a very simple Special Ordered Set (SOS) model.
# The model consists of 3 continuous variables, no linear constraints,
# and a pair of SOS constraints of type 1.

import gurobipy as gp
from gurobipy import GRB

try:
    # Create a new model

    model = gp.Model("sos")

    # Create variables

    x0 = model.addVar(ub=1.0, name="x0")
    x1 = model.addVar(ub=1.0, name="x1")
    x2 = model.addVar(ub=2.0, name="x2")

    # Set objective
    model.setObjective(2 * x0 + x1 + x2, GRB.MAXIMIZE)

    # Add first SOS: x0 = 0 or x1 = 0
    model.addSOS(GRB.SOS_TYPE1, [x0, x1], [1, 2])

    # Add second SOS: x0 = 0 or x2 = 0
    model.addSOS(GRB.SOS_TYPE1, [x0, x2], [1, 2])

    model.optimize()

    for v in model.getVars():
        print(f"{v.VarName} {v.X:g}")

    print(f"Obj: {model.ObjVal:g}")

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")

except AttributeError:
    print("Encountered an attribute error")
```

sudoku.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Sudoku example.

# The Sudoku board is a 9x9 grid, which is further divided into a 3x3 grid
# of 3x3 grids.  Each cell in the grid must take a value from 0 to 9.
# No two grid cells in the same row, column, or 3x3 subgrid may take the
# same value.
#
# In the MIP formulation, binary variables x[i,j,v] indicate whether
# cell <i,j> takes value 'v'.  The constraints are as follows:
#   1. Each cell must take exactly one value (sum_v x[i,j,v] = 1)
#   2. Each value is used exactly once per row (sum_i x[i,j,v] = 1)
#   3. Each value is used exactly once per column (sum_j x[i,j,v] = 1)
#   4. Each value is used exactly once per 3x3 subgrid (sum_grid x[i,j,v] = 1)
#
# Input datasets for this example can be found in examples/data/sudoku*.

import sys
import math
import gurobipy as gp
from gurobipy import GRB


if len(sys.argv) < 2:
    print("Usage: sudoku.py filename")
    sys.exit(0)

f = open(sys.argv[1])

grid = f.read().split()

n = len(grid[0])
s = int(math.sqrt(n))


# Create our 3-D array of model variables

model = gp.Model("sudoku")

vars = model.addVars(n, n, n, vtype=GRB.BINARY, name="G")


# Fix variables associated with cells whose values are pre-specified

for i in range(n):
    for j in range(n):
        if grid[i][j] != ".":
            v = int(grid[i][j]) - 1
```

(continues on next page)

(continued from previous page)

```
            vars[i, j, v].LB = 1

# Each cell must take one value

model.addConstrs(
    (vars.sum(i, j, "*") == 1 for i in range(n) for j in range(n)), name="V"
)

# Each value appears once per row

model.addConstrs(
    (vars.sum(i, "*", v) == 1 for i in range(n) for v in range(n)), name="R"
)

# Each value appears once per column

model.addConstrs(
    (vars.sum("*", j, v) == 1 for j in range(n) for v in range(n)), name="C"
)


# Each value appears once per subgrid

model.addConstrs(
    (
        gp.quicksum(
            vars[i, j, v]
            for i in range(i0 * s, (i0 + 1) * s)
            for j in range(j0 * s, (j0 + 1) * s)
        )
        == 1
        for v in range(n)
        for i0 in range(s)
        for j0 in range(s)
    ),
    name="Sub",
)

model.optimize()

model.write("sudoku.lp")

print("")
print("Solution:")
print("")

# Retrieve optimization result

solution = model.getAttr("X", vars)

for i in range(n):
    sol = ""
```

(continues on next page)

(continued from previous page)

```
    for j in range(n):
        for v in range(n):
            if solution[i, j, v] > 0.5:
                sol += str(v + 1)
    print(sol)
```

tsp.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Solve a traveling salesman problem on a randomly generated set of points
# using lazy constraints.  The base MIP model only includes 'degree-2'
# constraints, requiring each node to have exactly two incident edges.
# Solutions to this model may contain subtours - tours that don't visit every
# city.  The lazy constraint callback adds new constraints to cut them off.

import sys
import logging
import math
import random
from collections import defaultdict
from itertools import combinations

import gurobipy as gp
from gurobipy import GRB


def shortest_subtour(edges):
    """Given a list of edges, return the shortest subtour (as a list of nodes)
    found by following those edges. It is assumed there is exactly one 'in'
    edge and one 'out' edge for every node represented in the edge list."""

    # Create a mapping from each node to its neighbours
    node_neighbors = defaultdict(list)
    for i, j in edges:
        node_neighbors[i].append(j)
    assert all(len(neighbors) == 2 for neighbors in node_neighbors.values())

    # Follow edges to find cycles. Each time a new cycle is found, keep track
    # of the shortest cycle found so far and restart from an unvisited node.
    unvisited = set(node_neighbors)
    shortest = None
    while unvisited:
        cycle = []
        neighbors = list(unvisited)
        while neighbors:
            current = neighbors.pop()
            cycle.append(current)
```

(continues on next page)

(continued from previous page)

```
            unvisited.remove(current)
            neighbors = [j for j in node_neighbors[current] if j in unvisited]
        if shortest is None or len(cycle) < len(shortest):
            shortest = cycle

    assert shortest is not None
    return shortest


class TSPCallback:
    """Callback class implementing lazy constraints for the TSP.  At MIPSOL
    callbacks, solutions are checked for subtours and subtour elimination
    constraints are added if needed."""

    def __init__(self, nodes, x):
        self.nodes = nodes
        self.x = x

    def __call__(self, model, where):
        """Callback entry point: call lazy constraints routine when new
        solutions are found. Stop the optimization if there is an exception in
        user code."""
        if where == GRB.Callback.MIPSOL:
            try:
                self.eliminate_subtours(model)
            except Exception:
                logging.exception("Exception occurred in MIPSOL callback")
                model.terminate()

    def eliminate_subtours(self, model):
        """Extract the current solution, check for subtours, and formulate lazy
        constraints to cut off the current solution if subtours are found.
        Assumes we are at MIPSOL."""
        values = model.cbGetSolution(self.x)
        edges = [(i, j) for (i, j), v in values.items() if v > 0.5]
        tour = shortest_subtour(edges)
        if len(tour) < len(self.nodes):
            # add subtour elimination constraint for every pair of cities in tour
            model.cbLazy(
                gp.quicksum(self.x[i, j] for i, j in combinations(tour, 2))
                <= len(tour) - 1
            )


def solve_tsp(nodes, distances):
    """
    Solve a dense symmetric TSP using the following base formulation:

    min  sum_ij d_ij x_ij
    s.t. sum_j x_ij == 2   forall i in V
         x_ij binary       forall (i,j) in E
```

(continues on next page)

(continued from previous page)

```
    and subtours eliminated using lazy constraints.
    """

    with gp.Env() as env, gp.Model(env=env) as m:
        # Create variables, and add symmetric keys to the resulting dictionary
        # 'x', such that (i, j) and (j, i) refer to the same variable.
        x = m.addVars(distances.keys(), obj=distances, vtype=GRB.BINARY, name="e")
        x.update({(j, i): v for (i, j), v in x.items()})

        # Create degree 2 constraints
        for i in nodes:
            m.addConstr(gp.quicksum(x[i, j] for j in nodes if i != j) == 2)

        # Optimize model using lazy constraints to eliminate subtours
        m.Params.LazyConstraints = 1
        cb = TSPCallback(nodes, x)
        m.optimize(cb)

        # Extract the solution as a tour
        edges = [(i, j) for (i, j), v in x.items() if v.X > 0.5]
        tour = shortest_subtour(edges)
        assert set(tour) == set(nodes)

        return tour, m.ObjVal


if __name__ == "__main__":
    # Parse arguments
    if len(sys.argv) < 2:
        print("Usage: tsp.py npoints <seed>")
        sys.exit(0)
    npoints = int(sys.argv[1])
    seed = int(sys.argv[2]) if len(sys.argv) > 2 else 1

    # Create n random points in 2D
    random.seed(seed)
    nodes = list(range(npoints))
    points = [(random.randint(0, 100), random.randint(0, 100)) for i in nodes]

    # Dictionary of Euclidean distance between each pair of points
    distances = {
        (i, j): math.sqrt(sum((points[i][k] - points[j][k]) ** 2 for k in range(2)))
        for i, j in combinations(nodes, 2)
    }

    tour, cost = solve_tsp(nodes, distances)

    print("")
    print(f"Optimal tour: {tour}")
    print(f"Optimal cost: {cost:g}")
    print("")
```

tune.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

#  This example reads a model from a file and tunes it.
#  It then writes the best parameter settings to a file
#  and solves the model using these parameters.

import sys
import gurobipy as gp

if len(sys.argv) < 2:
    print("Usage: tune.py filename")
    sys.exit(0)

# Read the model
model = gp.read(sys.argv[1])

# Set the TuneResults parameter to 2
#
# The first parameter setting is the result for the first solved
# setting. The second entry the parameter setting of the best parameter
# setting.
model.Params.TuneResults = 2

# Tune the model
model.tune()

if model.TuneResultCount >= 2:
    # Load the best tuned parameters into the model
    #
    # Note, the first parameter setting is associated to the first solved
    # setting and the second parameter setting to best tune result.
    model.getTuneResult(1)

    # Write tuned parameters to a file
    model.write("tune.prm")

    # Solve the model using the tuned parameters
    model.optimize()
```

workforce1.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Assign workers to shifts; each worker may or may not be available on a
# particular day. If the problem cannot be solved, use IIS to find a set of
# conflicting constraints. Note that there may be additional conflicts besides
# what is reported via IIS.

import gurobipy as gp
from gurobipy import GRB
import sys

# Number of workers required for each shift
shifts, shiftRequirements = gp.multidict(
    {
        "Mon1": 3,
        "Tue2": 2,
        "Wed3": 4,
        "Thu4": 4,
        "Fri5": 5,
        "Sat6": 6,
        "Sun7": 5,
        "Mon8": 2,
        "Tue9": 2,
        "Wed10": 3,
        "Thu11": 4,
        "Fri12": 6,
        "Sat13": 7,
        "Sun14": 5,
    }
)

# Amount each worker is paid to work one shift
workers, pay = gp.multidict(
    {
        "Amy": 10,
        "Bob": 12,
        "Cathy": 10,
        "Dan": 8,
        "Ed": 8,
        "Fred": 9,
        "Gu": 11,
    }
)

# Worker availability
availability = gp.tuplelist(
    [
        ("Amy", "Tue2"),
        ("Amy", "Wed3"),
```

(continues on next page)

(continued from previous page)

```
        ("Amy", "Fri5"),
        ("Amy", "Sun7"),
        ("Amy", "Tue9"),
        ("Amy", "Wed10"),
        ("Amy", "Thu11"),
        ("Amy", "Fri12"),
        ("Amy", "Sat13"),
        ("Amy", "Sun14"),
        ("Bob", "Mon1"),
        ("Bob", "Tue2"),
        ("Bob", "Fri5"),
        ("Bob", "Sat6"),
        ("Bob", "Mon8"),
        ("Bob", "Thu11"),
        ("Bob", "Sat13"),
        ("Cathy", "Wed3"),
        ("Cathy", "Thu4"),
        ("Cathy", "Fri5"),
        ("Cathy", "Sun7"),
        ("Cathy", "Mon8"),
        ("Cathy", "Tue9"),
        ("Cathy", "Wed10"),
        ("Cathy", "Thu11"),
        ("Cathy", "Fri12"),
        ("Cathy", "Sat13"),
        ("Cathy", "Sun14"),
        ("Dan", "Tue2"),
        ("Dan", "Wed3"),
        ("Dan", "Fri5"),
        ("Dan", "Sat6"),
        ("Dan", "Mon8"),
        ("Dan", "Tue9"),
        ("Dan", "Wed10"),
        ("Dan", "Thu11"),
        ("Dan", "Fri12"),
        ("Dan", "Sat13"),
        ("Dan", "Sun14"),
        ("Ed", "Mon1"),
        ("Ed", "Tue2"),
        ("Ed", "Wed3"),
        ("Ed", "Thu4"),
        ("Ed", "Fri5"),
        ("Ed", "Sun7"),
        ("Ed", "Mon8"),
        ("Ed", "Tue9"),
        ("Ed", "Thu11"),
        ("Ed", "Sat13"),
        ("Ed", "Sun14"),
        ("Fred", "Mon1"),
        ("Fred", "Tue2"),
        ("Fred", "Wed3"),
        ("Fred", "Sat6"),
```

(continues on next page)

(continued from previous page)

```
        ("Fred", "Mon8"),
        ("Fred", "Tue9"),
        ("Fred", "Fri12"),
        ("Fred", "Sat13"),
        ("Fred", "Sun14"),
        ("Gu", "Mon1"),
        ("Gu", "Tue2"),
        ("Gu", "Wed3"),
        ("Gu", "Fri5"),
        ("Gu", "Sat6"),
        ("Gu", "Sun7"),
        ("Gu", "Mon8"),
        ("Gu", "Tue9"),
        ("Gu", "Wed10"),
        ("Gu", "Thu11"),
        ("Gu", "Fri12"),
        ("Gu", "Sat13"),
        ("Gu", "Sun14"),
    ]
)

# Model
m = gp.Model("assignment")

# Assignment variables: x[w,s] == 1 if worker w is assigned to shift s.
# Since an assignment model always produces integer solutions, we use
# continuous variables and solve as an LP.
x = m.addVars(availability, ub=1, name="x")

# The objective is to minimize the total pay costs
m.setObjective(gp.quicksum(pay[w] * x[w, s] for w, s in availability), GRB.MINIMIZE)

# Constraints: assign exactly shiftRequirements[s] workers to each shift s
reqCts = m.addConstrs((x.sum("*", s) == shiftRequirements[s] for s in shifts), "_")

# Using Python looping constructs, the preceding statement would be...
#
# reqCts = {}
# for s in shifts:
#   reqCts[s] = m.addConstr(
#         gp.quicksum(x[w,s] for w,s in availability.select('*', s)) ==
#         shiftRequirements[s], s)

# Save model
m.write("workforce1.lp")

# Optimize
m.optimize()
status = m.Status
if status == GRB.UNBOUNDED:
    print("The model cannot be solved because it is unbounded")
    sys.exit(0)
```

(continues on next page)

(continued from previous page)

```
if status == GRB.OPTIMAL:
    print(f"The optimal objective is {m.ObjVal:g}")
    sys.exit(0)
if status != GRB.INF_OR_UNBD and status != GRB.INFEASIBLE:
    print(f"Optimization was stopped with status {status}")
    sys.exit(0)

# do IIS
print("The model is infeasible; computing IIS")
m.computeIIS()
if m.IISMinimal:
    print("IIS is minimal\n")
else:
    print("IIS is not minimal\n")
print("\nThe following constraint(s) cannot be satisfied:")
for c in m.getConstrs():
    if c.IISConstr:
        print(c.ConstrName)
```

workforce2.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Assign workers to shifts; each worker may or may not be available on a
# particular day. If the problem cannot be solved, use IIS iteratively to
# find all conflicting constraints.

import gurobipy as gp
from gurobipy import GRB
import sys

# Number of workers required for each shift
shifts, shiftRequirements = gp.multidict(
    {
        "Mon1": 3,
        "Tue2": 2,
        "Wed3": 4,
        "Thu4": 4,
        "Fri5": 5,
        "Sat6": 6,
        "Sun7": 5,
        "Mon8": 2,
        "Tue9": 2,
        "Wed10": 3,
        "Thu11": 4,
        "Fri12": 6,
        "Sat13": 7,
        "Sun14": 5,
```

(continues on next page)

(continued from previous page)

```
    }
)

# Amount each worker is paid to work one shift
workers, pay = gp.multidict(
    {
        "Amy": 10,
        "Bob": 12,
        "Cathy": 10,
        "Dan": 8,
        "Ed": 8,
        "Fred": 9,
        "Gu": 11,
    }
)

# Worker availability
availability = gp.tuplelist(
    [
        ("Amy", "Tue2"),
        ("Amy", "Wed3"),
        ("Amy", "Fri5"),
        ("Amy", "Sun7"),
        ("Amy", "Tue9"),
        ("Amy", "Wed10"),
        ("Amy", "Thu11"),
        ("Amy", "Fri12"),
        ("Amy", "Sat13"),
        ("Amy", "Sun14"),
        ("Bob", "Mon1"),
        ("Bob", "Tue2"),
        ("Bob", "Fri5"),
        ("Bob", "Sat6"),
        ("Bob", "Mon8"),
        ("Bob", "Thu11"),
        ("Bob", "Sat13"),
        ("Cathy", "Wed3"),
        ("Cathy", "Thu4"),
        ("Cathy", "Fri5"),
        ("Cathy", "Sun7"),
        ("Cathy", "Mon8"),
        ("Cathy", "Tue9"),
        ("Cathy", "Wed10"),
        ("Cathy", "Thu11"),
        ("Cathy", "Fri12"),
        ("Cathy", "Sat13"),
        ("Cathy", "Sun14"),
        ("Dan", "Tue2"),
        ("Dan", "Wed3"),
        ("Dan", "Fri5"),
        ("Dan", "Sat6"),
        ("Dan", "Mon8"),
```

(continues on next page)

(continued from previous page)

```
        ("Dan", "Tue9"),
        ("Dan", "Wed10"),
        ("Dan", "Thu11"),
        ("Dan", "Fri12"),
        ("Dan", "Sat13"),
        ("Dan", "Sun14"),
        ("Ed", "Mon1"),
        ("Ed", "Tue2"),
        ("Ed", "Wed3"),
        ("Ed", "Thu4"),
        ("Ed", "Fri5"),
        ("Ed", "Sun7"),
        ("Ed", "Mon8"),
        ("Ed", "Tue9"),
        ("Ed", "Thu11"),
        ("Ed", "Sat13"),
        ("Ed", "Sun14"),
        ("Fred", "Mon1"),
        ("Fred", "Tue2"),
        ("Fred", "Wed3"),
        ("Fred", "Sat6"),
        ("Fred", "Mon8"),
        ("Fred", "Tue9"),
        ("Fred", "Fri12"),
        ("Fred", "Sat13"),
        ("Fred", "Sun14"),
        ("Gu", "Mon1"),
        ("Gu", "Tue2"),
        ("Gu", "Wed3"),
        ("Gu", "Fri5"),
        ("Gu", "Sat6"),
        ("Gu", "Sun7"),
        ("Gu", "Mon8"),
        ("Gu", "Tue9"),
        ("Gu", "Wed10"),
        ("Gu", "Thu11"),
        ("Gu", "Fri12"),
        ("Gu", "Sat13"),
        ("Gu", "Sun14"),
    ]
)

# Model
m = gp.Model("assignment")

# Assignment variables: x[w,s] == 1 if worker w is assigned to shift s.
# Since an assignment model always produces integer solutions, we use
# continuous variables and solve as an LP.
x = m.addVars(availability, ub=1, name="x")

# The objective is to minimize the total pay costs
m.setObjective(gp.quicksum(pay[w] * x[w, s] for w, s in availability), GRB.MINIMIZE)
```

(continues on next page)

(continued from previous page)

```
# Constraint: assign exactly shiftRequirements[s] workers to each shift s
reqCts = m.addConstrs((x.sum("*", s) == shiftRequirements[s] for s in shifts), "_")

# Optimize
m.optimize()
status = m.Status
if status == GRB.UNBOUNDED:
    print("The model cannot be solved because it is unbounded")
    sys.exit(0)
if status == GRB.OPTIMAL:
    print(f"The optimal objective is {m.ObjVal:g}")
    sys.exit(0)
if status != GRB.INF_OR_UNBD and status != GRB.INFEASIBLE:
    print(f"Optimization was stopped with status {status}")
    sys.exit(0)

# do IIS
print("The model is infeasible; computing IIS")
removed = []

# Loop until we reduce to a model that can be solved
while True:
    m.computeIIS()
    print("\nThe following constraint cannot be satisfied:")
    for c in m.getConstrs():
        if c.IISConstr:
            print(c.ConstrName)
            # Remove a single constraint from the model
            removed.append(str(c.ConstrName))
            m.remove(c)
            break
    print("")

    m.optimize()
    status = m.Status

    if status == GRB.UNBOUNDED:
        print("The model cannot be solved because it is unbounded")
        sys.exit(0)
    if status == GRB.OPTIMAL:
        break
    if status != GRB.INF_OR_UNBD and status != GRB.INFEASIBLE:
        print(f"Optimization was stopped with status {status}")
        sys.exit(0)

print("\nThe following constraints were removed to get a feasible LP:")
print(removed)
```

workforce3.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Assign workers to shifts; each worker may or may not be available on a
# particular day. If the problem cannot be solved, relax the model
# to determine which constraints cannot be satisfied, and how much
# they need to be relaxed.

import gurobipy as gp
from gurobipy import GRB
import sys

# Number of workers required for each shift
shifts, shiftRequirements = gp.multidict(
    {
        "Mon1": 3,
        "Tue2": 2,
        "Wed3": 4,
        "Thu4": 4,
        "Fri5": 5,
        "Sat6": 6,
        "Sun7": 5,
        "Mon8": 2,
        "Tue9": 2,
        "Wed10": 3,
        "Thu11": 4,
        "Fri12": 6,
        "Sat13": 7,
        "Sun14": 5,
    }
)

# Amount each worker is paid to work one shift
workers, pay = gp.multidict(
    {"Amy": 10, "Bob": 12, "Cathy": 10, "Dan": 8, "Ed": 8, "Fred": 9, "Gu": 11}
)

# Worker availability
availability = gp.tuplelist(
    [
        ("Amy", "Tue2"),
        ("Amy", "Wed3"),
        ("Amy", "Fri5"),
        ("Amy", "Sun7"),
        ("Amy", "Tue9"),
        ("Amy", "Wed10"),
        ("Amy", "Thu11"),
        ("Amy", "Fri12"),
        ("Amy", "Sat13"),
        ("Amy", "Sun14"),
```

(continues on next page)

(continued from previous page)

```
        ("Bob", "Mon1"),
        ("Bob", "Tue2"),
        ("Bob", "Fri5"),
        ("Bob", "Sat6"),
        ("Bob", "Mon8"),
        ("Bob", "Thu11"),
        ("Bob", "Sat13"),
        ("Cathy", "Wed3"),
        ("Cathy", "Thu4"),
        ("Cathy", "Fri5"),
        ("Cathy", "Sun7"),
        ("Cathy", "Mon8"),
        ("Cathy", "Tue9"),
        ("Cathy", "Wed10"),
        ("Cathy", "Thu11"),
        ("Cathy", "Fri12"),
        ("Cathy", "Sat13"),
        ("Cathy", "Sun14"),
        ("Dan", "Tue2"),
        ("Dan", "Wed3"),
        ("Dan", "Fri5"),
        ("Dan", "Sat6"),
        ("Dan", "Mon8"),
        ("Dan", "Tue9"),
        ("Dan", "Wed10"),
        ("Dan", "Thu11"),
        ("Dan", "Fri12"),
        ("Dan", "Sat13"),
        ("Dan", "Sun14"),
        ("Ed", "Mon1"),
        ("Ed", "Tue2"),
        ("Ed", "Wed3"),
        ("Ed", "Thu4"),
        ("Ed", "Fri5"),
        ("Ed", "Sun7"),
        ("Ed", "Mon8"),
        ("Ed", "Tue9"),
        ("Ed", "Thu11"),
        ("Ed", "Sat13"),
        ("Ed", "Sun14"),
        ("Fred", "Mon1"),
        ("Fred", "Tue2"),
        ("Fred", "Wed3"),
        ("Fred", "Sat6"),
        ("Fred", "Mon8"),
        ("Fred", "Tue9"),
        ("Fred", "Fri12"),
        ("Fred", "Sat13"),
        ("Fred", "Sun14"),
        ("Gu", "Mon1"),
        ("Gu", "Tue2"),
        ("Gu", "Wed3"),
```

(continues on next page)

(continued from previous page)

```
        ("Gu", "Fri5"),
        ("Gu", "Sat6"),
        ("Gu", "Sun7"),
        ("Gu", "Mon8"),
        ("Gu", "Tue9"),
        ("Gu", "Wed10"),
        ("Gu", "Thu11"),
        ("Gu", "Fri12"),
        ("Gu", "Sat13"),
        ("Gu", "Sun14"),
    ]
)

# Model
m = gp.Model("assignment")

# Assignment variables: x[w,s] == 1 if worker w is assigned to shift s.
# Since an assignment model always produces integer solutions, we use
# continuous variables and solve as an LP.
x = m.addVars(availability, ub=1, name="x")

# The objective is to minimize the total pay costs
m.setObjective(gp.quicksum(pay[w] * x[w, s] for w, s in availability), GRB.MINIMIZE)

# Constraint: assign exactly shiftRequirements[s] workers to each shift s
reqCts = m.addConstrs((x.sum("*", s) == shiftRequirements[s] for s in shifts), "_")

# Optimize
m.optimize()
status = m.Status
if status == GRB.UNBOUNDED:
    print("The model cannot be solved because it is unbounded")
    sys.exit(0)
if status == GRB.OPTIMAL:
    print(f"The optimal objective is {m.ObjVal:g}")
    sys.exit(0)
if status != GRB.INF_OR_UNBD and status != GRB.INFEASIBLE:
    print(f"Optimization was stopped with status {status}")
    sys.exit(0)

# Relax the constraints to make the model feasible
print("The model is infeasible; relaxing the constraints")
orignumvars = m.NumVars
m.feasRelaxS(0, False, False, True)
m.optimize()
status = m.Status
if status in (GRB.INF_OR_UNBD, GRB.INFEASIBLE, GRB.UNBOUNDED):
    print(
        "The relaxed model cannot be solved \
           because it is infeasible or unbounded"
    )
    sys.exit(1)
```

(continues on next page)

(continued from previous page)

```
if status != GRB.OPTIMAL:
    print(f"Optimization was stopped with status {status}")
    sys.exit(1)

print("\nSlack values:")
slacks = m.getVars()[orignumvars:]
for sv in slacks:
    if sv.X > 1e-6:
        print(f"{sv.VarName} = {sv.X:g}")
```

workforce4.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Assign workers to shifts; each worker may or may not be available on a
# particular day. We use lexicographic optimization to solve the model:
# first, we minimize the linear sum of the slacks. Then, we constrain
# the sum of the slacks, and we minimize a quadratic objective that
# tries to balance the workload among the workers.

import gurobipy as gp
from gurobipy import GRB
import sys

# Number of workers required for each shift
shifts, shiftRequirements = gp.multidict(
    {
        "Mon1": 3,
        "Tue2": 2,
        "Wed3": 4,
        "Thu4": 4,
        "Fri5": 5,
        "Sat6": 6,
        "Sun7": 5,
        "Mon8": 2,
        "Tue9": 2,
        "Wed10": 3,
        "Thu11": 4,
        "Fri12": 6,
        "Sat13": 7,
        "Sun14": 5,
    }
)

# Amount each worker is paid to work one shift
workers, pay = gp.multidict(
    {
```

(continues on next page)

(continued from previous page)

```
        "Amy": 10,
        "Bob": 12,
        "Cathy": 10,
        "Dan": 8,
        "Ed": 8,
        "Fred": 9,
        "Gu": 11,
    }
)

# Worker availability
availability = gp.tuplelist(
    [
        ("Amy", "Tue2"),
        ("Amy", "Wed3"),
        ("Amy", "Fri5"),
        ("Amy", "Sun7"),
        ("Amy", "Tue9"),
        ("Amy", "Wed10"),
        ("Amy", "Thu11"),
        ("Amy", "Fri12"),
        ("Amy", "Sat13"),
        ("Amy", "Sun14"),
        ("Bob", "Mon1"),
        ("Bob", "Tue2"),
        ("Bob", "Fri5"),
        ("Bob", "Sat6"),
        ("Bob", "Mon8"),
        ("Bob", "Thu11"),
        ("Bob", "Sat13"),
        ("Cathy", "Wed3"),
        ("Cathy", "Thu4"),
        ("Cathy", "Fri5"),
        ("Cathy", "Sun7"),
        ("Cathy", "Mon8"),
        ("Cathy", "Tue9"),
        ("Cathy", "Wed10"),
        ("Cathy", "Thu11"),
        ("Cathy", "Fri12"),
        ("Cathy", "Sat13"),
        ("Cathy", "Sun14"),
        ("Dan", "Tue2"),
        ("Dan", "Wed3"),
        ("Dan", "Fri5"),
        ("Dan", "Sat6"),
        ("Dan", "Mon8"),
        ("Dan", "Tue9"),
        ("Dan", "Wed10"),
        ("Dan", "Thu11"),
        ("Dan", "Fri12"),
        ("Dan", "Sat13"),
        ("Dan", "Sun14"),
```

(continues on next page)

(continued from previous page)

```
        ("Ed", "Mon1"),
        ("Ed", "Tue2"),
        ("Ed", "Wed3"),
        ("Ed", "Thu4"),
        ("Ed", "Fri5"),
        ("Ed", "Sun7"),
        ("Ed", "Mon8"),
        ("Ed", "Tue9"),
        ("Ed", "Thu11"),
        ("Ed", "Sat13"),
        ("Ed", "Sun14"),
        ("Fred", "Mon1"),
        ("Fred", "Tue2"),
        ("Fred", "Wed3"),
        ("Fred", "Sat6"),
        ("Fred", "Mon8"),
        ("Fred", "Tue9"),
        ("Fred", "Fri12"),
        ("Fred", "Sat13"),
        ("Fred", "Sun14"),
        ("Gu", "Mon1"),
        ("Gu", "Tue2"),
        ("Gu", "Wed3"),
        ("Gu", "Fri5"),
        ("Gu", "Sat6"),
        ("Gu", "Sun7"),
        ("Gu", "Mon8"),
        ("Gu", "Tue9"),
        ("Gu", "Wed10"),
        ("Gu", "Thu11"),
        ("Gu", "Fri12"),
        ("Gu", "Sat13"),
        ("Gu", "Sun14"),
    ]
)

# Model
m = gp.Model("assignment")

# Assignment variables: x[w,s] == 1 if worker w is assigned to shift s.
# This is no longer a pure assignment model, so we must use binary variables.
x = m.addVars(availability, vtype=GRB.BINARY, name="x")

# Slack variables for each shift constraint so that the shifts can
# be satisfied
slacks = m.addVars(shifts, name="Slack")

# Variable to represent the total slack
totSlack = m.addVar(name="totSlack")

# Variables to count the total shifts worked by each worker
totShifts = m.addVars(workers, name="TotShifts")
```

(continues on next page)

(continued from previous page)

```
# Constraint: assign exactly shiftRequirements[s] workers to each shift s,
# plus the slack
reqCts = m.addConstrs(
    (slacks[s] + x.sum("*", s) == shiftRequirements[s] for s in shifts), "_"
)

# Constraint: set totSlack equal to the total slack
m.addConstr(totSlack == slacks.sum(), "totSlack")

# Constraint: compute the total number of shifts for each worker
m.addConstrs((totShifts[w] == x.sum(w) for w in workers), "totShifts")

# Objective: minimize the total slack
# Note that this replaces the previous 'pay' objective coefficients
m.setObjective(totSlack)


# Optimize
def solveAndPrint():
    m.optimize()
    status = m.status
    if status in (GRB.INF_OR_UNBD, GRB.INFEASIBLE, GRB.UNBOUNDED):
        print(
            "The model cannot be solved because it is infeasible or \
               unbounded"
        )
        sys.exit(1)

    if status != GRB.OPTIMAL:
        print(f"Optimization was stopped with status {status}")
        sys.exit(0)

    # Print total slack and the number of shifts worked for each worker
    print("")
    print(f"Total slack required: {totSlack.X:g}")
    for w in workers:
        print(f"{w} worked {totShifts[w].X:g} shifts")
    print("")


solveAndPrint()

# Constrain the slack by setting its upper and lower bounds
totSlack.UB = totSlack.X
totSlack.LB = totSlack.X

# Variable to count the average number of shifts worked
avgShifts = m.addVar(name="avgShifts")

# Variables to count the difference from average for each worker;
# note that these variables can take negative values.
```

(continues on next page)

(continued from previous page)

```
diffShifts = m.addVars(workers, lb=-GRB.INFINITY, name="Diff")

# Constraint: compute the average number of shifts worked
m.addConstr(len(workers) * avgShifts == totShifts.sum(), "avgShifts")

# Constraint: compute the difference from the average number of shifts
m.addConstrs((diffShifts[w] == totShifts[w] - avgShifts for w in workers), "Diff")

# Objective: minimize the sum of the square of the difference from the
# average number of shifts worked
m.setObjective(gp.quicksum(diffShifts[w] * diffShifts[w] for w in workers))

# Optimize
solveAndPrint()
```

workforce5.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Assign workers to shifts; each worker may or may not be available on a
# particular day. We use multi-objective optimization to solve the model.
# The highest-priority objective minimizes the sum of the slacks
# (i.e., the total number of uncovered shifts). The secondary objective
# minimizes the difference between the maximum and minimum number of
# shifts worked among all workers.  The second optimization is allowed
# to degrade the first objective by up to the smaller value of 10% and 2 */

import gurobipy as gp
from gurobipy import GRB
import sys

# Sample data
# Sets of days and workers
Shifts = [
    "Mon1",
    "Tue2",
    "Wed3",
    "Thu4",
    "Fri5",
    "Sat6",
    "Sun7",
    "Mon8",
    "Tue9",
    "Wed10",
    "Thu11",
    "Fri12",
    "Sat13",
    "Sun14",
```

(continues on next page)

(continued from previous page)

```
]

Workers = ["Amy", "Bob", "Cathy", "Dan", "Ed", "Fred", "Gu", "Tobi"]

# Number of workers required for each shift
S = [3, 2, 4, 4, 5, 6, 5, 2, 2, 3, 4, 6, 7, 5]
shiftRequirements = {s: S[i] for i, s in enumerate(Shifts)}

# Worker availability: 0 if the worker is unavailable for a shift
A = [
    [0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1],
    [1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0],
    [0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1],
    [1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1],
    [0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1],
    [1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
]

availability = {
    (w, s): A[j][i] for i, s in enumerate(Shifts) for j, w in enumerate(Workers)
}

try:
    # Create model with a context manager. Upon exit from this block,
    # model.dispose is called automatically, and memory consumed by the model
    # is released.
    #
    # The model is created in the default environment, which will be created
    # automatically upon model construction.  For safe release of resources
    # tied to the default environment, disposeDefaultEnv is called below.
    with gp.Model("workforce5") as model:
        # Initialize assignment decision variables:
        # x[w][s] == 1 if worker w is assigned to shift s.
        # This is no longer a pure assignment model, so we must
        # use binary variables.
        x = model.addVars(
            availability.keys(), ub=availability, vtype=GRB.BINARY, name="x"
        )

        # Slack variables for each shift constraint so that the shifts can
        # be satisfied
        slacks = model.addVars(Shifts, name="Slack")

        # Variable to represent the total slack
        totSlack = model.addVar(name="totSlack")

        # Variables to count the total shifts worked by each worker
        totShifts = model.addVars(Workers, name="TotShifts")

        # Constraint: assign exactly shiftRequirements[s] workers
```

(continues on next page)

(continued from previous page)

```
        # to each shift s, plus the slack
        model.addConstrs(
            (x.sum("*", s) + slacks[s] == shiftRequirements[s] for s in Shifts),
            name="shiftRequirement",
        )

        # Constraint: set totSlack equal to the total slack
        model.addConstr(totSlack == slacks.sum(), name="totSlack")

        # Constraint: compute the total number of shifts for each worker
        model.addConstrs(
            (totShifts[w] == x.sum(w, "*") for w in Workers), name="totShifts"
        )

        # Constraint: set minShift/maxShift variable to less/greater than the
        # number of shifts among all workers
        minShift = model.addVar(name="minShift")
        maxShift = model.addVar(name="maxShift")
        model.addGenConstrMin(minShift, totShifts.values(), name="minShift")
        model.addGenConstrMax(maxShift, totShifts.values(), name="maxShift")

        # Set global sense for ALL objectives
        model.ModelSense = GRB.MINIMIZE

        # Set up primary objective
        model.setObjectiveN(
            totSlack, index=0, priority=2, abstol=2.0, reltol=0.1, name="TotalSlack"
        )

        # Set up secondary objective
        model.setObjectiveN(maxShift - minShift, index=1, priority=1, name="Fairness")

        # Save problem
        model.write("workforce5.lp")

        # Optimize
        model.optimize()

        status = model.Status

        if status in (GRB.INF_OR_UNBD, GRB.INFEASIBLE, GRB.UNBOUNDED):
            print("Model cannot be solved because it is infeasible or unbounded")
            sys.exit(0)

        if status != GRB.OPTIMAL:
            print(f"Optimization was stopped with status {status}")
            sys.exit(0)

        # Print total slack and the number of shifts worked for each worker
        print("")
        print(f"Total slack required: {totSlack.X}")
        for w in Workers:
```

(continues on next page)

(continued from previous page)

```
            print(f"{w} worked {totShifts[w].X} shifts")
        print("")

except gp.GurobiError as e:
    print(f"Error code {e.errno}: {e}")
except AttributeError as e:
    print(f"Encountered an attribute error: {e}")
finally:
    # Safely release memory and/or server side resources consumed by
    # the default environment.
    gp.disposeDefaultEnv()
```

workforce_batchmode.py

```
#!/usr/bin/env python3.11

# Copyright 2025, Gurobi Optimization, LLC

# Assign workers to shifts; each worker may or may not be available on a
# particular day.  The optimization problem is solved as a batch, and
# the schedule constructed only from the meta data available in the solution
# JSON.
#
# NOTE: You'll need a license file configured to use a Cluster Manager
#       for this example to run.

import time
import json
import sys
import gurobipy as gp
from gurobipy import GRB
from collections import OrderedDict, defaultdict


# For later pretty printing names for the shifts
shiftname = OrderedDict(
    [
        ("Mon1", "Monday 8:00"),
        ("Mon8", "Monday 14:00"),
        ("Tue2", "Tuesday 8:00"),
        ("Tue9", "Tuesday 14:00"),
        ("Wed3", "Wednesday 8:00"),
        ("Wed10", "Wednesday 14:00"),
        ("Thu4", "Thursday 8:00"),
        ("Thu11", "Thursday 14:00"),
        ("Fri5", "Friday 8:00"),
        ("Fri12", "Friday 14:00"),
        ("Sat6", "Saturday 8:00"),
        ("Sat13", "Saturday 14:00"),
        ("Sun7", "Sunday 9:00"),
```

(continues on next page)

(continued from previous page)

```
        ("Sun14", "Sunday 12:00"),
    ]
)


# Build the assignment problem in a Model, and submit it for batch optimization
#
# Required input: A Cluster Manager environment setup for batch optimization
def submit_assigment_problem(env):
    # Number of workers required for each shift
    shifts, shiftRequirements = gp.multidict(
        {
            "Mon1": 3,
            "Tue2": 2,
            "Wed3": 4,
            "Thu4": 4,
            "Fri5": 5,
            "Sat6": 5,
            "Sun7": 3,
            "Mon8": 2,
            "Tue9": 2,
            "Wed10": 3,
            "Thu11": 4,
            "Fri12": 5,
            "Sat13": 7,
            "Sun14": 5,
        }
    )

    # Amount each worker is paid to work one shift
    workers, pay = gp.multidict(
        {
            "Amy": 10,
            "Bob": 12,
            "Cathy": 10,
            "Dan": 8,
            "Ed": 8,
            "Fred": 9,
            "Gu": 11,
        }
    )

    # Worker availability
    availability = gp.tuplelist(
        [
            ("Amy", "Tue2"),
            ("Amy", "Wed3"),
            ("Amy", "Thu4"),
            ("Amy", "Sun7"),
            ("Amy", "Tue9"),
            ("Amy", "Wed10"),
            ("Amy", "Thu11"),
```

(continues on next page)

(continued from previous page)

```
            ("Amy", "Fri12"),
            ("Amy", "Sat13"),
            ("Amy", "Sun14"),
            ("Bob", "Mon1"),
            ("Bob", "Tue2"),
            ("Bob", "Fri5"),
            ("Bob", "Sat6"),
            ("Bob", "Mon8"),
            ("Bob", "Thu11"),
            ("Bob", "Sat13"),
            ("Cathy", "Wed3"),
            ("Cathy", "Thu4"),
            ("Cathy", "Fri5"),
            ("Cathy", "Sun7"),
            ("Cathy", "Mon8"),
            ("Cathy", "Tue9"),
            ("Cathy", "Wed10"),
            ("Cathy", "Thu11"),
            ("Cathy", "Fri12"),
            ("Cathy", "Sat13"),
            ("Cathy", "Sun14"),
            ("Dan", "Tue2"),
            ("Dan", "Thu4"),
            ("Dan", "Fri5"),
            ("Dan", "Sat6"),
            ("Dan", "Mon8"),
            ("Dan", "Tue9"),
            ("Dan", "Wed10"),
            ("Dan", "Thu11"),
            ("Dan", "Fri12"),
            ("Dan", "Sat13"),
            ("Dan", "Sun14"),
            ("Ed", "Mon1"),
            ("Ed", "Tue2"),
            ("Ed", "Wed3"),
            ("Ed", "Thu4"),
            ("Ed", "Fri5"),
            ("Ed", "Sat6"),
            ("Ed", "Mon8"),
            ("Ed", "Tue9"),
            ("Ed", "Thu11"),
            ("Ed", "Sat13"),
            ("Ed", "Sun14"),
            ("Fred", "Mon1"),
            ("Fred", "Tue2"),
            ("Fred", "Wed3"),
            ("Fred", "Sat6"),
            ("Fred", "Mon8"),
            ("Fred", "Tue9"),
            ("Fred", "Fri12"),
            ("Fred", "Sat13"),
            ("Fred", "Sun14"),
```

(continues on next page)

(continued from previous page)

```
            ("Gu", "Mon1"),
            ("Gu", "Tue2"),
            ("Gu", "Wed3"),
            ("Gu", "Fri5"),
            ("Gu", "Sat6"),
            ("Gu", "Sun7"),
            ("Gu", "Mon8"),
            ("Gu", "Tue9"),
            ("Gu", "Wed10"),
            ("Gu", "Thu11"),
            ("Gu", "Fri12"),
            ("Gu", "Sat13"),
            ("Gu", "Sun14"),
        ]
    )

    # Start environment, get model in this environment
    with gp.Model("assignment", env=env) as m:
        # Assignment variables: x[w,s] == 1 if worker w is assigned to shift s.
        # Since an assignment model always produces integer solutions, we use
        # continuous variables and solve as an LP.
        x = m.addVars(availability, ub=1, name="x")

        # Set tags encoding the assignments for later retrieval of the schedule.
        # Each tag is a JSON string of the format
        #   {
        #     "Worker": "<Name of the worker>",
        #     "Shift":  "String representation of the shift"
        #   }
        #
        for k, v in x.items():
            name, timeslot = k
            d = {"Worker": name, "Shift": shiftname[timeslot]}
            v.VTag = json.dumps(d)

        # The objective is to minimize the total pay costs
        m.setObjective(
            gp.quicksum(pay[w] * x[w, s] for w, s in availability), GRB.MINIMIZE
        )

        # Constraints: assign exactly shiftRequirements[s] workers to each shift
        reqCts = m.addConstrs(
            (x.sum("*", s) == shiftRequirements[s] for s in shifts), "_"
        )

        # Submit this model for batch optimization to the cluster manager
        # and return its batch ID for later querying the solution
        batchID = m.optimizeBatch()

    return batchID
```

(continues on next page)

(continued from previous page)

```
# Wait for the final status of the batch.
# Initially the status of a batch is "submitted"; the status will change
# once the batch has been processed (by a compute server).
def waitforfinalbatchstatus(batch):
    # Wait no longer than ten seconds
    maxwaittime = 10

    starttime = time.time()
    while batch.BatchStatus == GRB.BATCH_SUBMITTED:
        # Abort this batch if it is taking too long
        curtime = time.time()
        if curtime - starttime > maxwaittime:
            batch.abort()
            break

        # Wait for one second
        time.sleep(1)

        # Update the resident attribute cache of the Batch object with the
        # latest values from the cluster manager.
        batch.update()


# Print the schedule according to the solution in the given dict
def print_shift_schedule(soldict):
    schedule = defaultdict(list)

    # Iterate over the variables that take a non-zero value (i.e.,
    # an assignment), and collect them per day
    for v in soldict["Vars"]:
        # There is only one VTag, the JSON dict of an assignment we passed
        # in as the VTag
        assignment = json.loads(v["VTag"][0])
        schedule[assignment["Shift"]].append(assignment["Worker"])

    # Print the schedule
    for k in shiftname.values():
        day, time = k.split()
        workers = ", ".join(schedule[k])
        print(f" - {day:10} {time:>5}: {workers}")


if __name__ == "__main__":
    # Create Cluster Manager environment in batch mode.
    env = gp.Env(empty=True)
    env.setParam("CSBatchMode", 1)

    # env is a context manager; upon leaving, Env.dispose() is called
    with env.start():
        # Submit the assignment problem to the cluster manager, get batch ID
        batchID = submit_assigment_problem(env)
```

(continues on next page)